# A Third-Order Gaussian Process Trajectory Representation Framework With Closed-Form Kinematics for Continuous-Time Motion Estimation

Thien-Minh Nguyen, *Member, IEEE*, Ziyu Cao, *Student Member, IEEE*, Kailai Li, *Member, IEEE*, William Talbot, *Student Member, IEEE*, Tongxing Jin, Shenghai Yuan, *Member, IEEE*, Timothy D. Barfoot, *Fellow, IEEE*, and Lihua Xie, *Fellow, IEEE*

***Abstract*—In this article, we propose a third-order, i.e., white-noise-on-jerk, Gaussian process trajectory representation (TR) framework for continuous-time motion estimation (ME) tasks. Our framework features a unified TR that encapsulates the kinematic models of both $\mathbf{SO(3)} \times \mathbb{R}^3$ and $\mathbf{SE(3)}$ pose representations. This encapsulation strategy allows users to use the same implementation of measurement-based factors for either choice of pose representation, which facilitates experimentation and comparison to make a better choice for the ME task. In addition, unique to our framework, we derive the kinematic models with the *closed-form temporal derivatives of the local variables of* $\mathbf{SO(3)}$ *and* $\mathbf{SE(3)}$, which so far has only been approximated based on Taylor expansion in the literature. Our experiments show that these kinematic models can improve the estimation accuracy in high-speed scenarios. All analytical Jacobians of the interpolated states with respect to the support states of the TR, as well as the motion prior factors, are also provided for accelerated Gauss–Newton optimization. Our experiments demonstrate the efficacy and efficiency of the framework in various ME tasks, such as localization, calibration, and odometry, facilitating fast prototyping for ME researchers. We release the source code for the benefit of the community.**

***Index Terms*—Gaussian processes, state estimation, sensor fusion, manifolds, optimization methods.**

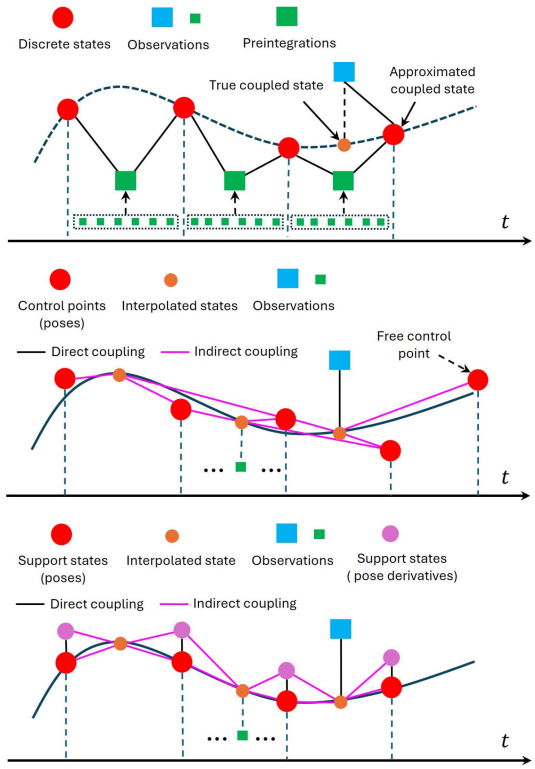



Fig. 1. In traditional DT TR (top), we need to approximate, perform extra processing, or use sophisticated hardware synchronization to restrict the coupling of the observations to a finite number of state estimates. In contrast, under the B-spline-based CT trajectory representation scheme (middle), direct coupling can be achieved via interpolation of control points, but they do not lie on the trajectory. Instead, GP-based representation uses *support states* that include trajectory poses with explicit temporal derivatives according to a chosen prior motion model.

## I. Introduction

CONTINUOUS-TIME (CT) motion estimation (ME) is a powerful technique that rectifies several disadvantages of the conventional discrete-time (DT) approach, the biggest of which is that it restricts the coupling of observations with a few chosen discrete states, leading to some undesirable compromises, as illustrated in Fig. 1. It allows one to directly fuse sensor measurements at the sample acquisition time. This is very useful for high-frequency and temporally spread-out sensors,

Received 11 June 2025; revised 20 February 2026; accepted 5 May 2026. Date of publication 9 June 2026; date of current version 6 July 2026. This work was supported by the National Research Foundation, Singapore, under its Medium Sized Centre for Advanced Robotics Technology Innovation. This article was recommended for publication by Associate Editor S. Weiss and Editor P. Robuffo Giordano upon evaluation of the reviewers' comments. *(Corresponding author: Thien-Minh Nguyen.)*

Thien-Minh Nguyen was with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798. He is now with the School of Mechanical and Mining Engineering, The University of Queensland, St Lucia, QLD 4072, Australia (e-mail: thienminh.nguyen@uq.edu.au).

Ziyu Cao is with the Department of Electrical Engineering, Linköping University, 58183 Linköping, Sweden.

Kailai Li is with the Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, 9747 AG Groningen, The Netherlands.

William Talbot is with Robotic Systems Lab (RSL), ETH Zürich, 8092 Zürich, Switzerland.

Tongxing Jin, Shenghai Yuan, and Lihua Xie are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798.

Timothy D. Barfoot is with the Autonomous Space Robotics Laboratory (ASRL), University of Toronto, Toronto, ON M3H5T6, Canada.

Code is available online at https://github.com/brytsknguyen/gptr.

Digital Object Identifier 10.1109/TRO.2026.3701957

1941-0468 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

such as inertial measurement unit (IMU), event camera, and light detection and ranging (LiDAR). Besides, CT trajectory representation also naturally allows the estimation of temporal offsets and querying the state at any time instance on the trajectory. We refer to [1] for a more comprehensive comparative study of CT and DT approaches for simultaneous localization and mapping (SLAM).

Despite its advantages, the CT paradigm remains "underutilized" according to a recent survey [2]. Several CT approaches have been proposed over the years. The simplest of which is the linear-interpolation (LI) representation that assumes a constant rate of change between consecutive poses [3], [4], [5], [6], [7]. However, this representation cannot represent derivative states; hence, they cannot fuse IMU data directly at their timestamp.

One option to address this issue is by using a smooth polynomial representation of poses, as in B-spline methods, which allows representing the derivative states by the polynomials of lower orders. B-spline methods have been extensively applied in the batch-optimization problems for multisensor calibration [8], [9], [10]. In recent years, B-spline has also been used for sliding-window-based continuoustime motion estimation (CTME) of mono/stereo visual odometry (VO) or visual-inertial odometry (VIO) [11], [12], [13], [14], [15], [16], [17], ultra-wideband (UWB) with IMU [18], and LiDAR odometry (LO) or LiDAR-inertial odometry (LIO) [19], [20], [21], [22]. However, we find that B-spline representation still has some limitations. First, the B-spline model dictates that the trajectory follows a polynomial constraint. Indeed, in *motion planning*, one can choose a polynomial as the trajectory, but when estimating the motion, we cannot assume that the trajectory must follow a specific model. A simple counterexample is the velocity of a falling object under air drag, $\mathbf{v}(t) = \mathbf{v}_{\text{terminal}} \times \tanh(\mathbf{g} \cdot t/\mathbf{v}_{\text{terminal}})$, which follows an exponential trajectory, instead of the polynomial. Second, because of the polynomial kinematic constraints, the control points are not the actual states on the trajectory, which makes the initialization of new control points an issue that requires attention. While directly duplicating the new control points by the last optimized ones is a common approach, a prediction scheme grounded on the kinematics model can be more accurate, and improves the convergence rate. Finally, the B-spline also requires several control points that extend beyond the end time of the trajectory, which can move freely and make the regression problem ill-posed.

In parallel with B-spline developments, CTME based on temporal Gaussian process (GP) is also active, which—although not exhaustively—can be traced through a series of works [23], [24], [25], [26], [27]. Specifically, in the first work [23], Seeger [28] adapted the concept of *GP regression* in machine learning literature to a batch SLAM problem by using Gauss–Newton (GN) optimization. They formulated the trajectory as a composition of (possibly infinitely) many temporal basis functions, then exploited the *kernel trick* in the inverse manner to ensure computational tractability. Then, the authors in [24] and [25] proved that the GP priors based on a class of linear-time-varying (LTV) stochastic differential equations (SDE) driven by white noise have exactly sparse inverse kernel matrices, resulting in linear computatinal complexity in the number of estimation times. Furthermore, they show that interpolation is an $O(1)$ operation. In their next work, Anderson and Barfoot [26], for the first time, demonstrated how GP-based CTME can be extended to manifold states, hence enabling the adoption of this technique in a wide class of robotics problems, which are called simultaneous trajectory estimation and mapping (STEAM). A comprehensive monograph on GP, as well as state estimation in general, can be found in [27].

In our opinion, GP-based TR offers some advantages compared to B-spline. Rather than subjectively assuming some type of kinematics, sparse GP methods often use a well-grounded kinematic model as the prior, and only assume the derivative of the highest order to be a Gaussian-distributed stochastic term. Based on the order of the derivative, we can refer to the GP model as *random velocity*, *random acceleration*, *random jerk*, also known as white-noise-on-velocity/acceleration/jerk, or WNOV, WNOA, WNOJ. As illustrated in Fig. 1, while B-spline encodes the information on velocity and acceleration via a sequence of *control points* under the interpolation rule, GP directly includes velocity and acceleration quantities in the *support states*. While retaining all the benefits of the B-spline approach, GP representation also avoids the issue of poorly constrained control points. In addition, as the GP support states are true states on the trajectory, initialization of new support states can be done easily with the state transition model. Moreover, the state interpolation process only involves the two adjacent support states, making the organization, query, and extension of the trajectory much more straightforward than in B-spline representation. Another advantage of the GP-based methods is that the covariance can also be propagated and interpolated just like the states. We refer readers to [29] for a more comprehensive comparison between GP and B-spline, whereas in this article, we will focus on the difference between GP-based approaches.

Similar to CT methods in general, we believe that the GP-based paradigm has been underutilized despite a mature theoretical foundation, which we suspect is due to the scarcity of open-sourced and instructional works compared to DT approaches. The most prominent existing open-sourced work to be mentioned is STEAM[1] [30]. To the best of the authors' knowledge, only STEAM_ICP [31], [32], [33] from the same group has followed up with an open-sourced application. Another example of open-source GP representation is ESGPR [34]. However, ESGPR only considers the calibration applications, where a 9 DOF vector-space GP is used to model the linear position, velocity, and acceleration states of a target. We also noted GPMP [35] and its applications [36], [37], [38]; however, these works focus on motion planning instead of ME.

By learning from previous works, we are motivated to develop a new software framework to boost the popularity of GP-based CTME methods, named Gaussian process trajectory representation (GPTR). In a sense, our work follows the spirit of STEAM with some new developments. Several important features are targeted. Among this, the choice of pose representation is a cornerstone of the framework since this choice depends upon how forces and torques act upon the object undergoing motion [39], [40] and can affect the performance of the ME scheme. However, in practice, one may not be entirely certain which pose representation is more suitable for their application. To mitigate this issue, we propose a unified trajectory representation for $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ that encapsulates the internal kinematics that are specific to each pose representation. This unified framework allows the implementation of the cost factors to stay the same regardless of the chosen pose representation, thus saving researchers the time to experiment with different models.

[1][Online]. Available: https://github.com/utiasASRL/steam.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

For the time interval between support states, which is called the *motion prior interval* (MPI) and denoted as $\Delta t \in \mathbb{R}^+$, we opt to space them uniformly and let it be a user-defined hyperparameter. This is a common choice in the literature, although strategies for nonuniform spacing could be investigated in future work, as has been done for B-splines [2].

Next, we choose a random jerk model (WNOJ) for the interpolation and motion model prior. This is a third-order model, which is useful in a practical sense, since it allows the direct inclusion of accelerometer measurements. The hyperparameters of this model, the noise characteristics $\boldsymbol{\Sigma}_{\boldsymbol{\nu}}, \boldsymbol{\Sigma}_{\boldsymbol{\gamma}} \in \mathbb{R}^{3\times3}$, are user-defined, since they vary by experiment. Some previous works have partially explored the third-order model, such as a hybrid $\mathrm{SO}(3)$-WNOA + $\mathbb{R}^3$-WNOJ [41] and approximated (AP) $\mathrm{SE}(3)$-WNOJ [42]. Our work stands out by the investigation of both the $\mathrm{SO}(3) \times \mathbb{R}^3$-WNOJ and $\mathrm{SE}(3)$-WNOJ models with closed-form (CF) kinematics under a unified TR. Experiment results show that the CF kinematic model can achieve higher accuracy in high-speed scenarios than the AP model. Moreover, in one specific scenario involving visual–inertial factors, we can observe faster convergence because of the CF kinematic model requiring fewer iterations than the AP kinematic model.

Besides the mathematical structure, we also pay attention to the practical side of our framework. Specifically, our library is implemented in a few C++ header files. We also integrate our framework with other popular libraries in robotics, such as Sophus,[2] Eigen[3] for manifold class abstraction, and Ceres Solver [43] for GN optimization, thus making the framework versatile for adoption into other projects. We also provide some minimal working examples of CTME with GPTR in various maximum likelihood estimation (MLE) and maximum a posteriori (MAP) optimization schemes. These examples involve LiDAR, IMU, UWB, and visual factors with analytical Jacobians that follow the standard of the Ceres Solver, and are packaged under both ROS1 and ROS2 frameworks. These practical implementation approaches have been carefully deliberated to ensure the ease of adoption by the community.

The main contributions of our work can be stated as follows.

1) We propose a unified TR that encapsulates the internal kinematics of $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ to mitigate the difficulty of having to reimplement the ME scheme for each representation, thus facilitating experimentation and comparison to achieve the best model for the ME task.
2) We provide the CF kinematics of third-order on-manifold GPs based on both $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ pose representations.
3) We compare the performance of the proposed CF kinematics model with the existing AP model in various scenarios, demonstrating the benefits of the CF model.
4) We release our code base, named GPTR, consisting of our GP-based CTME framework and several working examples of MLE and MAP optimization problems with visual, UWB, IMU, and LiDAR factors. These examples can serve as prototypes to boost the applications of CTME and GP methods.

The rest of this article is organized as follows. Section II presents the related works. Section III recalls the basic theory of GP-based ME, i.e., propagation and interpolation on vector space. Section IV provides the detailed derivation of CF kinematics of GPs on both $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ manifolds. Section V establishes a trajectory representation that encapsulate the distinct kinematics of $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ pose representations. Section VI details several MLE and MAP optimization schemes leveraging image, IMU, UWB, and LiDAR measurements. Section VII presents the experimental results that showcase the accuracy and computation loads of the new CF equations, as well as the importance of selecting the pose representations. Finally, Section VIII concludes this article.

[2][Online]. Available: https://github.com/strasdat/Sophus.
[3][Online]. Available: https://eigen.tuxfamily.org/.

## II. Related Works

We briefly summarize the developments of CTME in relation to this article below.

*CTME for robotics:* CTME addresses key limitations of DT estimation by fusing measurements at acquisition time and supporting CT querying and time-offset estimation. Comparative discussions and surveys can be found in [1] and [2]. CTME has been validated in modern high-data-rate sensing pipelines. Early applications of CTME focused on the calibration of visual–inertial systems [8], [9], [10]. This was followed by pioneering work on LiDAR–inertial calibration in [44], which also introduced observability-aware analyses to improve efficiency and robustness under degenerative conditions. Building upon these foundations, subsequent studies developed multimodal SLAM systems [20], [32], [41], [45], [46], [47], integrating various combinations of inertial, visual, LiDAR, and radar modalities. Despite these benefits and a mature theoretical foundation, CTME remains underutilized in practice due to the higher implementation barrier and the scarcity of lightweight, instructional, and reusable toolchains compared to DT pipelines [2].

*CTME parameterization:* LI is a simple CT representation used in several CT estimation systems [3], [4], [5], [7], but it does not explicitly represent derivative states, which limits direct IMU fusion at its timestamps. B-spline trajectories provide smoothness and efficient interpolation and have been widely used for batch calibration and estimation [8], [9], [10], and later adopted in sliding-window approaches [16], [19], [20]. A common practical issue is that spline control points are not true trajectory states, which require attention to initialization and boundary handling.

*Prior-driven CTME:* GP-based CTME models the trajectory with an SDE-driven GP prior, building on GP regression views [23], [28] and exploiting sparse inverse kernels for scalable optimization and constant-time interpolation [24], [25]. Extensions to manifold states enable broad robotics adoption under the STEAM formulation [26], with comprehensive treatments in [27] and recent GP versus spline comparisons [29]. For higher order priors on pose manifolds, prior works often rely on hybrid representations or approximations [41], [42], motivating a revisit to kinematic and Jacobian CF, and their study on their benefits.

## III. Preliminaries

### A. Notations

In this article, a matrix is denoted by a bold uppercase letter and vectors are denoted by a bold lowercase letter. For a matrix $\boldsymbol{A}$ (and similarly, for vectors), $\boldsymbol{A}^\top$ denotes its transpose. Given $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, we also write $\boldsymbol{A} = [a_{ij}]$, where $a_{ij}$ refers to the entry at the $i$th row and $j$th column, where $i = 0, \ldots m-1$ and $j = 0, \ldots n-1$. For two matrices $\boldsymbol{A} \in \mathbb{R}^{a\times b}$ and $\boldsymbol{B} \in \mathbb{R}^{c\times d}$, $\boldsymbol{A} \otimes \boldsymbol{B} \in \mathbb{R}^{ac\times bd}$ denotes their Kronecker

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

product. For a number of square matrices $\{\boldsymbol{A}_i\}$ (including scalars), $\text{diag}(\dots \boldsymbol{A}_i \dots)$ denotes the block-diagonal matrix constructed from $\{\boldsymbol{A}_i\}$. Similarly, for $\{\boldsymbol{A}_i \in \mathbb{R}^3\}$, we denote $\text{vstack}(\dots \boldsymbol{A}_i \dots) \triangleq [\dots \boldsymbol{A}_i^\top \dots]^\top$ or $(\dots \boldsymbol{A}_i \dots)$ when the meaning is clear from the context.

To avoid ambiguity, we may attach a left superscript to indicate the *parent* coordinate frame and a left subscript for the *child* frame. Thus, ${}^{\mathtt{W}}_{\mathtt{B}}\mathbf{R}$ denotes the orientation of the frame B (body) w.r.t. the frame W (world). Moreover, for a vector $\mathbf{v}$, ${}^{\mathtt{B}}\mathbf{v}$ and ${}^{\mathtt{W}}\mathbf{v}$ explicitly refer to the coordinates of $\mathbf{v}$ in the coordinate frame B and W, respectively, and ${}^{\mathtt{W}}\mathbf{v} = {}^{\mathtt{W}}_{\mathtt{B}}\mathbf{R}\,{}^{\mathtt{B}}\mathbf{v}$. Similarly, we define a transformation matrix ${}^{\mathtt{W}}_{\mathtt{B}}\mathbf{T} = \left[\begin{smallmatrix} {}^{\mathtt{W}}_{\mathtt{B}}\mathbf{R} & {}^{\mathtt{W}}_{\mathtt{B}}\mathbf{p} \\ \mathbf{0} & 1 \end{smallmatrix}\right] \in \text{SE}(3)$, where $\mathbf{R} \in \text{SO}(3)$ and $\mathbf{p} \in \mathbb{R}^3$. The coordinate transform of a point $\mathbf{x}$ is ${}^{\mathtt{W}}\mathbf{x} = {}^{\mathtt{W}}_{\mathtt{B}}\mathbf{R}\,{}^{\mathtt{B}}\mathbf{x} + {}^{\mathtt{W}}_{\mathtt{B}}\mathbf{p}$. When the frames are variable, to simplify notations, we may supplant the full notation of the frame with the index or time stamp. For example ${}^{\mathtt{B}}_{\mathtt{S}_i}\mathbf{T} = {}_i\mathbf{T}$ can denote the *extrinsics* of the sensor $\mathtt{S}_i$ in the body frame. For a vector $\mathbf{x} \in \mathbb{R}^3$, $\mathbf{x}^\wedge$ denotes the corresponding skew-symmetric matrix. We overload the functions Exp and Log for the mappings between $\text{SO}(3)$ and $\text{SE}(3)$ with their vector tangents, i.e., $\text{Exp}(\cdot) : \mathbb{R}^n \to \mathcal{M}$ and $\text{Log}(\cdot) : \mathcal{M} \to \mathbb{R}^n$, $n = 3$ if $\mathcal{M} = \text{SO}(3)$ and $n = 6$ if $\mathcal{M} = \text{SE}(3)$. In addition, we follow [48] for the generalization of Jacobian from vector space to Jacobian on manifold, based on the right convention for plus/minus operations, as explained in [48, Sect. E].

We use $t$ to denote the time. $t$ can be included as an argument of a time-varying variable $\boldsymbol{x}$, i.e., $\boldsymbol{x}(t)$, or a subscript $\boldsymbol{x}_t$, or omitted when the context is clear. $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ denotes a multivariate normal distribution with mean $\boldsymbol{\mu}$, covariance $\boldsymbol{\Sigma}$, and $\mathcal{GP}(\boldsymbol{\mu}(t), \boldsymbol{\kappa}(t))$, which denotes a GP with the mean function $\boldsymbol{\mu}(t)$ and the covariance function $\boldsymbol{\kappa}(t)$.

### B. GP Propagation

Denote $\boldsymbol{x}(t)$ as a state vector of interest. Let us assume that $\boldsymbol{x}(t)$ evolves by the following LTV-SDE:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{A}(t)\boldsymbol{x}(t) + \boldsymbol{B}(t)\boldsymbol{\eta}(t),\ \boldsymbol{\eta}(t) \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}). \tag{1}$$

Also, we define $\boldsymbol{\Phi}(t, t_0)$ as the *state transition matrix* of the deterministic system

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{A}(t)\boldsymbol{x}(t),\ \boldsymbol{x}(t_0) = \boldsymbol{x}_0. \tag{2}$$

From (1), we have $\boldsymbol{x}(t) \sim \mathcal{GP}(\boldsymbol{\mu}(t), \boldsymbol{\kappa}(t, t_0))$ with the *mean function* $\boldsymbol{\mu}(t)$ and *covariance function* $\boldsymbol{\kappa}(t, t_0)$ defined as

$$\begin{aligned} \boldsymbol{\mu}(t) &= \boldsymbol{\Phi}(t, t_0)\mu(t_0) \\ \boldsymbol{\kappa}(t, t_0) &= \boldsymbol{\Phi}(t, t_0)\boldsymbol{\kappa}(t_0, t_0)\boldsymbol{\Phi}(t, t_0)^\top + \boldsymbol{Q}(t, t_0) \\ \boldsymbol{Q}(t, t_0) &= \int_{t_0}^{t} \boldsymbol{\Phi}(t, s)\boldsymbol{B}(s)\boldsymbol{\Sigma}\boldsymbol{B}(s)^\top \boldsymbol{\Phi}(t, s)^\top ds. \end{aligned} \tag{3}$$

### C. GP Interpolation

Given the samples of $\boldsymbol{\mu}(t)$ at different times $\{\boldsymbol{\mu}_k = \boldsymbol{\mu}(t_k)\}_{k=0,1,\dots,K}$, and assume that $\boldsymbol{x}(t)$ evolves according to (1), the authors in [24] and [27] showed that the mean state at time $\tau$, for $\tau \in [t_k, t_{k+1})$, can be found by

$$\boldsymbol{\mu}(\tau) = \boldsymbol{\Lambda}(\tau)\boldsymbol{\mu}_k + \boldsymbol{\Psi}(\tau)\boldsymbol{\mu}_{k+1} \tag{4}$$

where $\boldsymbol{\Lambda}(\tau)$ and $\boldsymbol{\Psi}(\tau)$ are defined as

$$\boldsymbol{\Lambda}(\tau) = \boldsymbol{\Phi}(\tau, t_k) - \boldsymbol{\Psi}(\tau)\boldsymbol{\Phi}(t_{k+1}, t_k) \tag{5}$$

$$\boldsymbol{\Psi}(\tau) = \boldsymbol{Q}(\tau, t_k)\boldsymbol{\Phi}(t_{k+1}, \tau)^\top \boldsymbol{Q}(t_{k+1}, t_k)^{-1}. \tag{6}$$

### D. GP on Vector Space

Given $\boldsymbol{\nu}(t) = (\mathbf{p}(t), \mathbf{v}(t), \mathbf{a}(t))$, where $\mathbf{p}, \mathbf{v}$, and $\mathbf{a}$ are the position, velocity, and acceleration, and assume a random *jerk* model, we have a GP based on the following linear time-invariant system:

$$\dot{\boldsymbol{\nu}}(t) = \boldsymbol{A}\boldsymbol{\nu}(t) + \boldsymbol{B}\boldsymbol{\eta}_{\boldsymbol{\nu}}(t),\ \boldsymbol{\eta}_{\boldsymbol{\nu}}(t) \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\boldsymbol{\nu}}) \tag{7}$$

where $\boldsymbol{A} \triangleq \begin{bmatrix} \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$ and $\boldsymbol{B} \triangleq \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{I} \end{bmatrix}$. We then define the transition matrix and the covariance matrix as follows:

$$\begin{aligned} \boldsymbol{F}(\Delta t) &\triangleq \boldsymbol{\Phi}(\Delta t, 0) = e^{\boldsymbol{A}\Delta t} \\ \boldsymbol{Q}_{\boldsymbol{\nu}}(\Delta t, \boldsymbol{\Sigma}_{\boldsymbol{\nu}}) &\triangleq \int_0^{\Delta t} \boldsymbol{F}(s)\boldsymbol{B}\boldsymbol{\Sigma}_{\boldsymbol{\nu}}\boldsymbol{B}^\top \boldsymbol{F}(s)^\top ds. \end{aligned} \tag{8}$$

Hence, we can obtain the discrete model

$$\begin{aligned} \boldsymbol{\nu}_{k+1} &= \boldsymbol{F}(\Delta t)\boldsymbol{\nu}_k + \boldsymbol{\eta}_k \triangleq \left[\bar{\boldsymbol{F}}(\Delta t) \otimes \mathbf{I}\right] \boldsymbol{\nu}_k + \boldsymbol{\eta}_k \\ \boldsymbol{\eta}_k &\sim \mathcal{N}(\mathbf{0}, \boldsymbol{Q}_{\boldsymbol{\nu}}) \triangleq \mathcal{N}(\mathbf{0}, \bar{\boldsymbol{Q}}_{\boldsymbol{\nu}}(\Delta t) \otimes \boldsymbol{\Sigma}_{\boldsymbol{\nu}}) \end{aligned} \tag{9}$$

where $\bar{\boldsymbol{F}}(\Delta t)$ and $\bar{\boldsymbol{Q}}_{\boldsymbol{\nu}}(\Delta t)$ can be quickly calculated by

$$\begin{aligned} \bar{\boldsymbol{F}}(\Delta t) &= [f_{nm}],\ f_{nm} = \begin{cases} \Delta t^m/m!, & \text{if } m \geq n \\ 0, & \text{otherwise} \end{cases} \\ \bar{\boldsymbol{Q}}_{\boldsymbol{\nu}}(\Delta t) &= [q_{nm}] \\ q_{nm} &= \frac{\Delta t^{2D+1-n-m}}{(2D+1-n-m)(D-n)!(D-m)!} \\ D &= N-1;\ n, m \in \{0, \dots N-1\}, \Delta t = t - t_0 \end{aligned} \tag{10}$$

where $N$ is the order of the system, which, for the specific case of $N = 3$, we have $\bar{\boldsymbol{F}}$ and $\bar{\boldsymbol{Q}}$ as

$$\bar{\boldsymbol{F}}(\Delta t) = \begin{bmatrix} 1 & 1 & 0.5\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 0 \end{bmatrix} \tag{11}$$

$$\bar{\boldsymbol{Q}}(\Delta t) = \begin{bmatrix} \Delta t^5/20 & \Delta t^4/8 & \Delta t^3/6 \\ \Delta t^4/8 & \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^3/6 & \Delta t^2/2 & \Delta t \end{bmatrix}. \tag{12}$$

*Remark 1:* The explicit forms of $\bar{\boldsymbol{F}}$ and $\bar{\boldsymbol{Q}}$ for models of second and third orders have been frequently presented in the existing literature on GP [24], [29], [34], [41], [42]. In this article, we stipulate the formulas (10) that can generate $\bar{\boldsymbol{F}}$ and $\bar{\boldsymbol{Q}}$ to both second-order and third-order models. These formulas are directly derived from the theoretical foundation of [24] and [42] for WNOA and WNOJ model, but expressed as implementation-friendly functions of $\Delta t$ and $N > 1$ and the indices of the matrix entries, thus waiving the need of having to calculate the matrix exponential and integral (8) for every $N$.

## IV. GP on Manifolds

In the previous section, we have reviewed the basics of GPs on vector space. However, in most applications, the trajectory must involve manifold states with significantly more complex

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DERIVATIONS OF THE KINEMATIC MODELS FOR GP ON $\mathrm{SO}(3)$ MANIFOLD

| **Approximated Approach** [29], [42] | **Closed-form Approach (Ours)** |
|---|---|
| **Global $\rightarrow$ Local**: | |
| $\boldsymbol{\theta} = \mathrm{Log}(\mathbf{R}_k^{-1}\mathbf{R})$ (13) | |
| $\dot{\boldsymbol{\theta}} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\omega},$ (14) | |
| $\ddot{\boldsymbol{\theta}} = \frac{d\dot{\boldsymbol{\theta}}}{dt} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\frac{d\boldsymbol{\omega}}{dt} + \frac{d}{dt}\left[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\right]\boldsymbol{\omega}$ $\approx \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\alpha} + \frac{d}{dt}\left[\mathbf{I} + \frac{1}{2}\boldsymbol{\theta}^{\wedge}\right]\boldsymbol{\omega}$ $= \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\omega}^{\wedge}\dot{\boldsymbol{\theta}}$ (15) | $\ddot{\boldsymbol{\theta}} = \frac{d\dot{\boldsymbol{\theta}}}{dt} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\frac{d\boldsymbol{\omega}}{dt} + \frac{\partial}{\partial\boldsymbol{\theta}}\left[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\omega}\right]\frac{d\boldsymbol{\theta}}{dt},$ $\triangleq \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\alpha} + \boldsymbol{H}_1'(\boldsymbol{\theta},\boldsymbol{\omega})\dot{\boldsymbol{\theta}},$ (16) $\boldsymbol{H}_1'(\boldsymbol{u},\boldsymbol{v}) = \frac{\partial \boldsymbol{J}_r^{-1}(\boldsymbol{u})\boldsymbol{v}}{\partial \boldsymbol{u}}$ (17) |
| **Global $\leftarrow$ Local**: | |
| $\mathbf{R} = \mathbf{R}_k\mathrm{Exp}(\boldsymbol{\theta})$ (18) | |
| $\boldsymbol{\omega} = \frac{\partial}{\partial\boldsymbol{\theta}}\left[\mathbf{R}_k\mathrm{Exp}(\boldsymbol{\theta})\right]\frac{d\boldsymbol{\theta}}{dt} = \boldsymbol{J}_r(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$ (19) | |
| $\boldsymbol{\alpha} \overset{(15)}{\approx} \boldsymbol{J}_r(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \frac{1}{2}\boldsymbol{J}_r(\boldsymbol{\theta})\boldsymbol{\omega}^{\wedge}\dot{\boldsymbol{\theta}}$ (20) | $\boldsymbol{\alpha} = \frac{d\boldsymbol{\omega}}{dt} = \boldsymbol{J}_r(\boldsymbol{\theta})\frac{d\dot{\boldsymbol{\theta}}}{dt} + \frac{\partial}{\partial\boldsymbol{\theta}}\left[\boldsymbol{J}_r(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}\right]\frac{d\boldsymbol{\theta}}{dt}$ $\triangleq \boldsymbol{J}_r(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}},$ (21) $\boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v}) \triangleq \frac{\partial \boldsymbol{J}_r(\boldsymbol{u})\boldsymbol{v}}{\partial \boldsymbol{u}}$ (22) |

Note: $\boldsymbol{J}_r$ is the *right Jacobian of* $\mathrm{SO}(3)$ and $\boldsymbol{J}_r^{-1}$ is its inverse [48]. In Appendix A, we detail the procedure to obtain the closed forms of the mappings $\boldsymbol{J}_r$, $\boldsymbol{J}_r^{-1}$, $\boldsymbol{H}_1(\cdot,\cdot)$, $\boldsymbol{H}_1'(\cdot,\cdot)$. These procedures will also be used as illustrations for a similar process in the $\mathrm{SE}(3)$ case.

kinematics. Thus, in this section, we shall detail the formulation of the third-order GP on two special manifolds of particular interest to ME.

### A. GP With $\mathrm{SO}(3) \times \mathbb{R}^3$ SO3 Pose Representation

Also called the split representation, in this formulation, two GPs are interpolated and propagated separately. In [41], a composition of a second-order GP for $\mathrm{SO}(3)$ and a third-order GP for $\mathbb{R}^3$ was proposed. In this section, we will detail the formulation of the third-order GP of $\mathrm{SO}(3)$, which will also provide some foundation for the $\mathrm{SE}(3)$ GP.

Let us define the state $(\mathbf{R}_t,\ \boldsymbol{\omega}_t,\ \boldsymbol{\alpha}_t) \in \mathrm{SO}(3) \times \mathbb{R}^3 \times \mathbb{R}^3$, where $\mathbf{R}_t$, $\boldsymbol{\omega}_t$, and $\boldsymbol{\alpha}_t$ are, respectively, the *rotation*, *angular velocity*, and *angular acceleration* states. The evolution of $(\mathbf{R}_t,\ \boldsymbol{\omega}_t,\ \boldsymbol{\alpha}_t)$ can be modeled by the following process:

$$\dot{\mathbf{R}}_t = \mathbf{R}_t\boldsymbol{\omega}_t^{\wedge},\ \dot{\boldsymbol{\omega}}_t = \boldsymbol{\alpha}_t,\ \dot{\boldsymbol{\alpha}}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}). \tag{23}$$

However, (23) is nonlinear, and no CF solution of the mean and covariance of this process is known to the best of the authors' knowledge. To resolve this issue, for each interval $[t_k, t_{k+1})$, Anderson and Barfoot [26] defined the *local states* $\boldsymbol{\theta}_t$ and $\boldsymbol{\gamma}_t$ as

$$\boldsymbol{\theta}_t \triangleq \mathrm{Log}(\mathbf{R}_k^{-1}\mathbf{R}_t) \in \mathbb{R}^3,\ \boldsymbol{\gamma}_t = (\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, \ddot{\boldsymbol{\theta}}_t) \in \mathbb{R}^9 \tag{24}$$

which belong to a vector space, and we can assume that $\boldsymbol{\gamma}_t$ evolves by the same system as (7)

$$\dot{\boldsymbol{\gamma}}_t = \boldsymbol{A}\boldsymbol{\gamma}_t + \boldsymbol{B}\boldsymbol{\eta}_{\boldsymbol{\gamma}}(t),\ \boldsymbol{\eta}_{\boldsymbol{\gamma}}(t) \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\boldsymbol{\gamma}}). \tag{25}$$

By the model (25), propagation and interpolation of the rotation state and its derivatives can be performed on flat Euclidean space $\mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3$ just like (7). The kinematic models of the global states $(\mathbf{R}_t, \boldsymbol{\omega}_t, \boldsymbol{\alpha}_t)$ and local states $(\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, \ddot{\boldsymbol{\theta}}_t)$ are derived in Table I, first by using the AP method [29], [42], and then with our new CF approach. In Remark 2, we will further discuss the difference between AP and CF models.

### B. GP With $\mathrm{SE}(3)$ Pose Representation

The derivation of kinematic models for $\mathrm{SE}(3)$ representation follows a similar procedure to the $\mathrm{SO}(3) \times \mathbb{R}^3$ case in Table I, so we skip the side-by-side comparison in this section. Before detailing the derivation, let us recall some basic properties of $\mathrm{SE}(3)$. Let $\boldsymbol{\xi} = (\boldsymbol{\theta}, \boldsymbol{\rho})$ be the vector form of the Lie-algebra of $\mathrm{SE}(3)$, where $\boldsymbol{\theta} \in \mathbb{R}^3$ is the rotation vector and $\boldsymbol{\rho} \in \mathbb{R}^3$ is the translational part. The mappings $\mathrm{Exp} : \mathbb{R}^6 \rightarrow \mathrm{SE}(3)$ and $\mathrm{Log} : \mathrm{SE}(3) \rightarrow \mathbb{R}^6$ are defined as [27]

$$\mathrm{Exp}(\boldsymbol{\xi}) \triangleq \begin{bmatrix} \mathrm{Exp}(\boldsymbol{\theta}) & \mathbf{V}(\boldsymbol{\theta})\boldsymbol{\rho} \\ \mathbf{0} & 1 \end{bmatrix},\ \mathrm{Log}(\mathrm{Exp}(\boldsymbol{\xi})) = \begin{bmatrix} \boldsymbol{\theta} \\ \boldsymbol{\rho} \end{bmatrix} \tag{26}$$

where $\mathbf{V}(\boldsymbol{\theta}) = \boldsymbol{J}_r(-\boldsymbol{\theta})$ [48].

Hence, between the time instances $t_k$ and $t_{k+1}$, we can convert between the local and global states in the same manner as Section IV-A. Specifically given the global states $\mathbf{T}$, $\boldsymbol{\tau}$, and $\dot{\boldsymbol{\tau}}$, we can find the local state $\boldsymbol{\xi}$, $\dot{\boldsymbol{\xi}}$, and $\ddot{\boldsymbol{\xi}}$ as

$$\boldsymbol{\xi} \triangleq \mathrm{Log}(\mathbf{T}_k^{-1}\mathbf{T}) \tag{27}$$

$$\dot{\boldsymbol{\xi}} \triangleq \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\boldsymbol{\tau} \tag{28}$$

$$\ddot{\boldsymbol{\xi}} = \frac{d\dot{\boldsymbol{\xi}}}{dt} = \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\frac{d\boldsymbol{\tau}}{dt} + \frac{\partial}{\partial\boldsymbol{\xi}}\left[\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\boldsymbol{\tau}\right]\frac{d\boldsymbol{\xi}}{dt} \triangleq \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\dot{\boldsymbol{\tau}} + \boldsymbol{\mathcal{H}}_1'(\boldsymbol{\xi},\boldsymbol{\tau})\dot{\boldsymbol{\xi}} \tag{29}$$

and from local states, we find the global states

$$\mathbf{T} \triangleq \mathbf{T}_k\mathrm{Exp}(\boldsymbol{\xi}) \tag{30}$$

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

$$\boldsymbol{\tau} = \boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\dot{\boldsymbol{\xi}} \tag{31}$$

$$\begin{aligned}\dot{\boldsymbol{\tau}} = \frac{d\boldsymbol{\tau}}{dt} &= \boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\frac{d\dot{\boldsymbol{\xi}}}{dt} + \frac{\partial}{\partial \boldsymbol{\xi}}\left[\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\dot{\boldsymbol{\xi}}\right]\frac{d\boldsymbol{\xi}}{dt} \\ &\triangleq \boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\ddot{\boldsymbol{\xi}} + \boldsymbol{\mathcal{H}}_1(\boldsymbol{\xi},\dot{\boldsymbol{\xi}})\dot{\boldsymbol{\xi}}\end{aligned} \tag{32}$$

where $\boldsymbol{\tau} = (\boldsymbol{\omega}, \boldsymbol{\nu})$ denotes the *twist* and $\dot{\boldsymbol{\tau}} = (\boldsymbol{\alpha}, \boldsymbol{\beta})$ is its derivative, and $\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})$ is the right Jacobian of $\mathrm{SE}(3)$ and $\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})$ is its inverse $\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})$. The CF of $\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})$, $\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})$, $\boldsymbol{\mathcal{H}}_1(\boldsymbol{\xi},\dot{\boldsymbol{\xi}})$, and $\boldsymbol{\mathcal{H}}_1'(\boldsymbol{\xi},\boldsymbol{\tau})$ are defined as follows:

$$\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi}) \triangleq \begin{bmatrix}\boldsymbol{J}_r(\boldsymbol{\theta}) & \mathbf{0} \\ \boldsymbol{Q}(\boldsymbol{\xi}) & \boldsymbol{J}_r(\boldsymbol{\theta})\end{bmatrix} \tag{33}$$

$$\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi}) = \begin{bmatrix}\boldsymbol{J}_r^{-1}(\boldsymbol{\theta}) & \mathbf{0} \\ \boldsymbol{Q}'(\boldsymbol{\xi}) & \boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\end{bmatrix} \tag{34}$$

$$\begin{aligned}\boldsymbol{\mathcal{H}}_1(\boldsymbol{\xi},\dot{\boldsymbol{\xi}}) &= \frac{\partial}{\partial \boldsymbol{\xi}}\left[\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\dot{\boldsymbol{\xi}}\right] \\ &= \begin{bmatrix}\frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{J}_r(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}\right] & \mathbf{0} \\ \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{Q}(\boldsymbol{\xi})\dot{\boldsymbol{\theta}}\right] + \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{J}_r(\boldsymbol{\theta})\dot{\boldsymbol{\rho}}\right] & \frac{\partial}{\partial \boldsymbol{\rho}}\left[\boldsymbol{Q}(\boldsymbol{\xi})\dot{\boldsymbol{\theta}}\right]\end{bmatrix} \\ &= \begin{bmatrix}\boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\theta}}) & \mathbf{0} \\ \boldsymbol{S}_1(\boldsymbol{\xi},\dot{\boldsymbol{\theta}}) + \boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\rho}}) & \boldsymbol{S}_2(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\end{bmatrix}\end{aligned} \tag{35}$$

$$\begin{aligned}\boldsymbol{\mathcal{H}}_1'(\boldsymbol{\xi},\boldsymbol{\tau}) &= \frac{\partial}{\partial \boldsymbol{\xi}}\left[\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\boldsymbol{\tau}\right] \\ &= \begin{bmatrix}\frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\omega}\right] & \mathbf{0} \\ \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{Q}'(\boldsymbol{\xi})\boldsymbol{\omega}\right] + \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{\nu}\right] & \frac{\partial}{\partial \boldsymbol{\rho}}\left[\boldsymbol{Q}'(\boldsymbol{\xi})\boldsymbol{\omega}\right]\end{bmatrix} \\ &= \begin{bmatrix}\boldsymbol{H}_1'(\boldsymbol{\theta},\boldsymbol{\omega}) & \mathbf{0} \\ \boldsymbol{S}_1'(\boldsymbol{\xi},\boldsymbol{\omega}) + \boldsymbol{H}_1'(\boldsymbol{\theta},\boldsymbol{\nu}) & \boldsymbol{S}_2'(\boldsymbol{\xi},\boldsymbol{\omega})\end{bmatrix}\end{aligned} \tag{36}$$

where $\boldsymbol{Q}'(\boldsymbol{\xi}) \triangleq -\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})\boldsymbol{Q}(\boldsymbol{\xi})\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})$, and $\boldsymbol{Q}(\boldsymbol{\xi})$ is detailed by Barfoot [27] in (8.91b) in. To keep this article self-contained, we find an alternative form of $\boldsymbol{Q}(\boldsymbol{\xi})$ as follows:

$$\boldsymbol{Q}(\boldsymbol{\xi}) = \boldsymbol{Q}(\boldsymbol{\theta},\boldsymbol{\rho}) = -\mathrm{Exp}(-\boldsymbol{\theta})\boldsymbol{H}_1(-\boldsymbol{\theta},\boldsymbol{\rho}) \tag{37}$$

and $\boldsymbol{S}_1$, $\boldsymbol{S}_2$, $\boldsymbol{S}_1'$, and $\boldsymbol{S}_2'$ are explained in Section IV-C.

In contrast, Tang et al. [42] previously used the Taylor expansion $\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi}) \approx \mathbf{I} + \frac{1}{2}\boldsymbol{\xi}^{\curlywedge}$ to approximate (29) and (32)

$$\ddot{\boldsymbol{\xi}} \approx \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{\xi})\dot{\boldsymbol{\tau}} + \frac{1}{2}\dot{\boldsymbol{\xi}}^{\curlywedge}\boldsymbol{\tau} \tag{38}$$

$$\dot{\boldsymbol{\tau}} \approx \boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\ddot{\boldsymbol{\xi}} - \frac{1}{2}\boldsymbol{\mathcal{J}}_r(\boldsymbol{\xi})\dot{\boldsymbol{\xi}}^{\curlywedge}\boldsymbol{\tau}. \tag{39}$$

*Remark 2:* The approximations (38) and (39) are close to the CF if $\boldsymbol{\xi}$ and $\dot{\boldsymbol{\xi}}$ (or $\boldsymbol{\theta}$ and $\dot{\boldsymbol{\theta}}$ for $\mathrm{SO}(3)$) are small within the $\Delta t$ interval, as remarked in [29] and [42]. This also informs us on when the two models would significantly diverge, i.e., in high-speed scenarios that entail larger relative motion in an MPI. In our framework, these approximations are also invoked when $\|\boldsymbol{\xi}\|$ or $\|\boldsymbol{\theta}\|$ is below a threshold to avoid numerical singularities. The AP models can also be kept in effect at all times by a user-defined boolean parameter. When configured to use the AP model with $\mathrm{SE}(3)$ pose representation, the GPTR framework becomes equivalent to STEAM, except for some small implementation details (STEAM uses the left Jacobian convention, where ours uses right; GPTR uses the Ceres solver for GN optimization, while STEAM has a custom solver). In Section VII, we shall conduct several experiments to demonstrate the benefit of the CF kinematic models over the AP model based on $\mathrm{SE}(3)$, which stands in for the comparison with STEAM, as well as the AP kinematic model for $\mathrm{SO}(3) \times \mathbb{R}^3$ that was first formulated in [29].

*Remark 3:* We notice in a very recent work, Barfoot [49] also derived a formula for derivatives in the kinematic models of general Lie groups based on some basic building blocks in the form of infinite series. In this article, we take a different approach to derive the CF of (16), (21), (29), and (32) by directly calculating the time derivatives of $\dot{\boldsymbol{\theta}}, \boldsymbol{\omega}$, $\dot{\boldsymbol{\xi}}$, and $\boldsymbol{\tau}$ from their already known CF, following the procedure in Section IV-C. We also conduct experiments to establish the advantage of the CF model.

### C. CF Derivative Procedure

The calculations of $\boldsymbol{S}_1, \boldsymbol{S}_2, \boldsymbol{S}_1'$, and $\boldsymbol{S}_2'$ in Section IV-B as well as $\boldsymbol{H}_1$ and $\boldsymbol{H}_1'$ in Section IV-A all follow a similar procedure. We detail the calculation of $\boldsymbol{H}_1$ and $\boldsymbol{H}_1'$ in Appendix A as an illustration of the procedure. However, due to the significantly larger number of computations involved in $\boldsymbol{S}_1$, $\boldsymbol{S}_2$, $\boldsymbol{S}_1'$, and $\boldsymbol{S}_2'$, we forgo the explicit details and opt to delineate the procedure underlying these calculations.

Specifically, we notice that the CF of $\mathrm{Exp}(\boldsymbol{\theta})$, $\boldsymbol{J}_r(\boldsymbol{\theta})$, $\boldsymbol{J}_r^{-1}(\boldsymbol{\theta})$, $\boldsymbol{Q}(\boldsymbol{\theta},\boldsymbol{\rho})$, and $\boldsymbol{Q}'(\boldsymbol{\theta},\boldsymbol{\rho})$ all have a similar structure as follows:

$$\boldsymbol{M} = \sum_{i=1}^{M} f_i(\boldsymbol{\theta},\boldsymbol{\rho})\, g_i(\|\boldsymbol{\theta}\|) \tag{40}$$

where $g_i(\cdot)$ consists of power and trigonometric terms of $\|\boldsymbol{\theta}\|$ [see (60)–(68) for example], and $f_i(\cdot,\cdot)$ are matrix polynomials of $\boldsymbol{\theta}^{\wedge}$ and $\boldsymbol{\rho}^{\wedge}$ [see (55)]. Given the CF of $g_i$ and $f_i$, for a vector $\boldsymbol{w}$, calculating the CF Jacobian of $\boldsymbol{M}\boldsymbol{w}$ is equivalent to finding the sum of the Jacobians on each $fwg$ term. Let us take the calculation of $\boldsymbol{S}_1'(\boldsymbol{\xi},\boldsymbol{\omega}) = \frac{\partial}{\partial \boldsymbol{\theta}}[\boldsymbol{Q}'(\boldsymbol{\theta},\boldsymbol{\rho})\boldsymbol{\omega}]$ in (36) as an illustrative example

$$\begin{aligned}\boldsymbol{S}_1' = \sum_{i=1}^{36} \big[&f_{iu}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})g_i(u) \\ &+ f_i(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}g_i^{(1)}(u)\bar{\boldsymbol{u}}^{\top}\big]\Big|_{\boldsymbol{u}=\boldsymbol{\theta},\boldsymbol{v}=\boldsymbol{\rho},\boldsymbol{w}=\boldsymbol{\omega}}\end{aligned} \tag{41}$$

where $u \triangleq \|\boldsymbol{u}\|$ and $\bar{\boldsymbol{u}} \triangleq \boldsymbol{u}/u$, $g_i^{(n)}(u) \triangleq \frac{\partial^n g_i(u)}{\partial u^n}$, and $f_{iu}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) \triangleq \frac{\partial}{\partial \boldsymbol{u}}[f_i(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}]$.

Due to the amount of $fg$ terms involved in $\boldsymbol{Q}(\boldsymbol{\xi})$ and $\boldsymbol{Q}'(\boldsymbol{\xi})$ and the page constraint, we shall not detail the CF of $g_i^{(n)}$ and $f_{iu}$ as with the $\mathrm{SO}(3) \times \mathbb{R}^3$ case in Appendix A. We refer the readers to our source code for these details.

## V. Unified GP Trajectory Representation

As explained earlier, when switching between $\mathbf{T}$ between and $(\mathbf{R},\mathbf{p})$, one has to reimplement all the factors in each ME task based on the selected representation. Obviously, this process is far from desirable, especially if one seeks to benchmark between different representations. Hence, in this section, we propose a trajectory representation scheme that allows users to build the same factors for the ME tasks that are invariant to the underlying kinematics.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

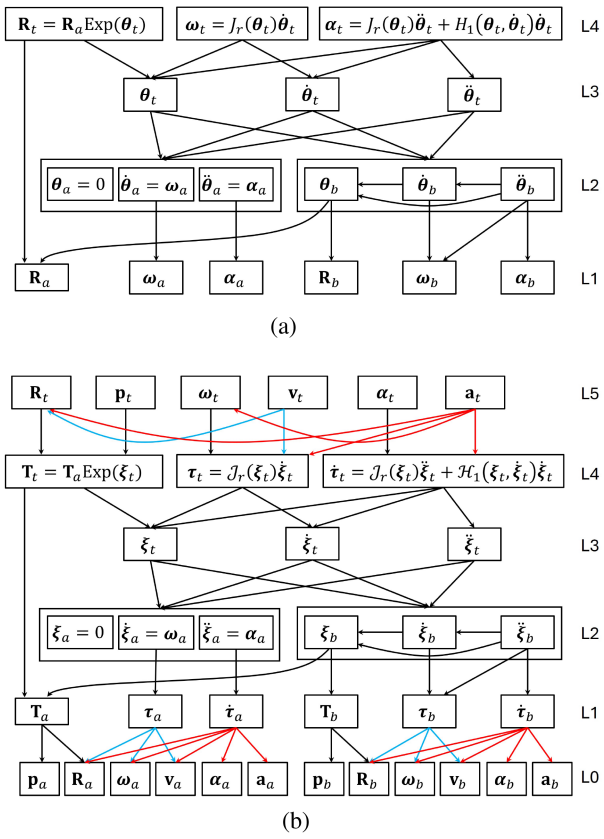



Fig. 2. Dependence graphs describing the hierarchies of variables for $\mathrm{SO}(3)$ GP [see Fig. 2(a)] and $\mathrm{SE}(3)$ GP [see Fig. 2(b)]. (a) Hierarchy of manifold variables in $\mathrm{SO}(3)$ GP. We group the variables into four levels, and the goal is to find the Jacobian of variables in level L4 w.r.t to variables in level L1. Every arrow from variable $v_i$ to $v_j$ represents a nonzero partial derivative $\mathbf{J}_{v_j}^{v_i}$. To avoid clustering, every arrow that goes into the subgroup $\gamma_a$ or $\gamma_b$ represents three actual arrows, each pointing to each variable in the subgroup. The intrinsic Jacobians describing these dependencies are detailed in Appendix C1. (b) Hierarchy of manifold variables in the $\mathrm{SE}(3)$ GP. We use color to help distinguish some variables with many dependencies. The structure is the same from L4 to L1, only that we now have new variables in L5 and L0. The Jacobians between L5 and L4 are detailed in (115)–(119), and the Jacobians between L1 and L0 are detailed in (122)–(126) of Appendix C2. The L5-L4 and L1-L0 Jacobians are the key to the unified trajectory representation in this work.

### A. Trajectory Representation

A CT trajectory represented by GP can be defined by the following three sets of user-defined parameters.

1) The MPI $\Delta t \in \mathbb{R}^+$.
2) A set of support states $\{\hat{\mathcal{X}}_k\}_{k=0}^K$, where $\hat{\mathcal{X}}_k \triangleq (\hat{\mathbf{R}}_k, \hat{\boldsymbol{\omega}}_k, \hat{\boldsymbol{\alpha}}_k, \hat{\mathbf{p}}_k, \hat{\mathbf{v}}_k, \hat{\mathbf{a}}_k) \in \mathrm{SO}(3) \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3$. Each support state represents the state estimates at the time instance $t_k = t_0 + k\Delta t$.
3) The covariance matrices $\boldsymbol{\Sigma}_{\boldsymbol{\nu}}, \boldsymbol{\Sigma}_{\boldsymbol{\gamma}} \in \mathbb{R}^{3\times 3}$ as defined in (7) and (25).

If $\mathrm{SO}(3) \times \mathbb{R}^3$ GP is chosen, then we assume that the *rotational states* $\mathbf{R}_k, \boldsymbol{\omega}_k, \boldsymbol{\alpha}_k$ evolve by the model (25), and the translational states $\mathbf{p}_k, \mathbf{v}_k,$ and $\mathbf{a}_k$ follow (7). On the other hand, if the $\mathrm{SE}(3)$ GP is chosen, then we can reshuffle the states internally to form the global states $\mathbf{T}_k, \boldsymbol{\tau}_k,$ and $\dot{\boldsymbol{\tau}}_k$ and assume that the local states $\boldsymbol{\xi}, \dot{\boldsymbol{\xi}},$ and $\ddot{\boldsymbol{\xi}}$ evolve by (7). This encapsulation strategy is visualized in Fig. 2. Based on the choice of the kinematic model, the interpolation and *intrinsic Jacobians* are calculated accordingly without affecting the ME factors.

*Remark 4:* Our choice of $(\hat{\mathbf{R}}_k, \hat{\boldsymbol{\omega}}_k, \hat{\boldsymbol{\alpha}}_k, \hat{\mathbf{p}}_k, \hat{\mathbf{v}}_k, \hat{\mathbf{a}}_k)$ as the basic state representation comes from the experience that $\mathbf{R}$, $\boldsymbol{\omega}$, $\mathbf{p}$, and $\mathbf{v}$ can often be directly measured by sensors, and thus more intuitive, whereas $\mathbf{T}_k$ and $\boldsymbol{\tau}_k$ appear to be some recombinations of $\mathbf{R}_k, \boldsymbol{\omega}_k, \mathbf{p}_k,$ and $\mathbf{v}_k$. In fact, we observe that the $\mathbf{R}$ state is the source of all complexities regarding state estimation. In theory, we can also use $(\hat{\mathbf{T}}_k, \hat{\boldsymbol{\tau}}_k, \hat{\dot{\boldsymbol{\tau}}}_k)$ as the basic state representation and $(\hat{\mathbf{R}}_k, \hat{\boldsymbol{\omega}}_k, \hat{\boldsymbol{\alpha}}_k, \hat{\mathbf{p}}_k, \hat{\mathbf{v}}_k, \hat{\mathbf{a}}_k)$ as the internally shuffled states.

### B. Regression

Given an observation or sensor measurement $\mathcal{Z}$ that is coupled with the state $\mathcal{X}_t$ via some observation model $h(\mathcal{Z}, \mathcal{X}_t, \ldots, \eta) = 0$, where $\eta$ represents the noise or uncertainty. The regression problem can be formalized as

$$\mathrm{argmin}_{\{\hat{\mathcal{X}}_m\}_{m=0}^K} \sum_{\mathcal{Z}} \left\| r(\mathcal{Z}_t, \hat{\mathcal{X}}_t, \ldots) \right\|_{\boldsymbol{W}}^2 \tag{42}$$

where $\|r\|_{\boldsymbol{W}}^2 \triangleq r^\top \boldsymbol{W} r$, $r \triangleq h(\mathcal{Z}, \hat{\mathcal{X}}_t, \ldots, \eta)|_{\eta=0}$ is the residual, and $\boldsymbol{W}$ is a weight based on the inverse of the covariance of $\eta$ (also known as the information matrix). Crucial to the solving of (42), one needs to calculate the Jacobian of the residual $r(\mathcal{Z}, \hat{\mathcal{X}}_t, \ldots)$ over the states estimate $\hat{\mathcal{X}}_a$ and $\hat{\mathcal{X}}_b$ where $[t_a,\ t_b] \ni t$. By using the chain rule, we have

$$\frac{\partial r}{\partial \hat{\mathcal{X}}_a} = \frac{\partial r}{\partial \hat{\mathcal{X}}_t}\frac{\partial \hat{\mathcal{X}}_t}{\partial \hat{\mathcal{X}}_a}, \quad \frac{\partial r}{\partial \hat{\mathcal{X}}_b} = \frac{\partial r}{\partial \hat{\mathcal{X}}_t}\frac{\partial \hat{\mathcal{X}}_t}{\partial \hat{\mathcal{X}}_b}. \tag{43}$$

The Jacobian $\frac{\partial r}{\partial \hat{\mathcal{X}}_t}$ depends on each specific residual. However, $\frac{\partial \hat{\mathcal{X}}_t}{\partial \hat{\mathcal{X}}_a}$ and $\frac{\partial \hat{\mathcal{X}}_t}{\partial \hat{\mathcal{X}}_b}$, which we call the *intrinsic Jacobians*, follow specific formulas based on the underlying pose representation ($\mathrm{SO}(3) \times \mathbb{R}^3$ or $\mathrm{SE}(3)$), as shown in Fig. 2. Calculating the CF of these Jacobians can be quite involved. However, they can substantially reduce computational cost compared to the automatic differentiation approach, sometimes up to 80% in some of our tests. We provide a systematic procedure for these calculations in Appendix C, believing that the process can inspire a deeper understanding of Lie algebra. We have also verified our derivations by using the automatic derivative feature in Ceres, which uses the jet number representation for finding the derivative of a function.[4]

### C. Motion Prior

Between every two consecutive support states $\hat{\mathcal{X}}_a$ and $\hat{\mathcal{X}}_b$ on the trajectory (we denote $a = k$ and $b = k + 1$ for simpler notations), we add a motion prior factor that is specialized for the underlying pose representation. This factor is crucial for the convergence of the CTME problem based on GP, as it constrains the feasible trajectories to those conforming to the dynamic models (7) and (25), effectively acting as a smoothing factor. For $\mathrm{SO}(3) \times \mathbb{R}^3$, we define the residual $\mathbf{r}_{\mathcal{M}}$ and the weight $\boldsymbol{W}_{\mathcal{M}}$ as follows:

$$\mathbf{r}_{\mathcal{M}} = \begin{bmatrix} \mathbf{r}_\gamma \\ \mathbf{r}_\nu \end{bmatrix} \triangleq \begin{bmatrix} \hat{\boldsymbol{\gamma}}_b - \boldsymbol{F}(\Delta t)\hat{\boldsymbol{\gamma}}_a \\ \hat{\boldsymbol{\nu}}_b - \boldsymbol{F}(\Delta t)\hat{\boldsymbol{\nu}}_a \end{bmatrix}$$

[4][Online]. Available: http://ceres-solver.org/automatic_derivatives.html.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

$$\boldsymbol{W}_{\mathcal{M}} = \begin{bmatrix} \boldsymbol{Q}_{\boldsymbol{\gamma}}(\Delta t) & \mathbf{0} \\ \mathbf{0} & \boldsymbol{Q}_{\boldsymbol{\nu}}(\Delta t) \end{bmatrix}^{-1}.$$

Similarly, for SE(3) representation, we define the residual as

$$\mathbf{r}_{\mathcal{M}} \triangleq \hat{\boldsymbol{\xi}}_b - \left[\bar{\boldsymbol{F}}(\Delta t) \otimes \mathbf{I}_6\right] \hat{\boldsymbol{\xi}}_a,\ \boldsymbol{W}_{\mathcal{M}} = \boldsymbol{Q}_{\boldsymbol{\xi}}(\Delta t)^{-1}.$$

As the motion prior is specialized for the chosen pose representation, we also provide an overloaded function to help the user quickly construct the factors without having to calculate the Jacobians for each type of motion prior above.

## VI. GP-Based CTME Prototypes

In this section, we will, respectively, present the general formulation of GP-based CTME schemes for visual, ranging, and LiDAR scenarios in Sections VI-A–VI-C.

### A. Visual–Inertial ME

In this first example, we demonstrate the application of GP-based CTME in calibrating visual–inertial sensor suites. Let us define the state estimates as follows:

$$\hat{\mathcal{T}} \triangleq \left(\ldots, \hat{\mathcal{X}}_m, \ldots, \hat{\mathbf{b}}_a, \hat{\mathbf{b}}_\omega, \hat{\mathbf{g}}, {}^{\mathtt{I}}_{\mathtt{C}_0}\hat{\mathbf{T}}, {}^{\mathtt{I}}_{\mathtt{C}_1}\hat{\mathbf{T}}\right) \tag{44}$$

$$\hat{\mathcal{X}}_m = (\hat{\mathbf{R}}_m, \hat{\boldsymbol{\omega}}_m, \hat{\boldsymbol{\alpha}}_m, \hat{\mathbf{p}}_m, \hat{\mathbf{v}}_m, \hat{\mathbf{a}}_m) \tag{45}$$

where $\hat{\mathbf{b}}_a$ and $\hat{\mathbf{b}}_\omega$ are accelerometer and gyroscope biases, respectively. $\hat{\mathcal{X}}_m$ are the support states of the trajectory estimate of the IMU sensor. $\hat{\mathbf{g}}$ denotes the estimated gravity vector. $\{{}^{\mathtt{I}}_{\mathtt{C}_i}\hat{\mathbf{T}}\}_{i=0}^{1}$ are the transformation from camera $i$ to IMU frame, i.e., the extrinsics. Hence, the ME and calibration using known landmarks in a batchwise manner are formulated as the minimization of the following cost function:

$$f(\hat{\mathcal{T}}) = \sum_{(\hat{\mathcal{X}}_k, \hat{\mathcal{X}}_{k+1})} \|\mathbf{r}_{\mathcal{M}}\|^2_{\mathbf{W}_{\mathcal{M}}} + \sum_{\mathcal{C}_t} \|\mathbf{r}_{\mathcal{C}}\|^2_{\mathbf{W}_{\mathcal{C}}} + \sum_{\mathcal{I}_t} \|\mathbf{r}_{\mathcal{I}}\|^2_{\mathbf{W}_{\mathcal{I}}} \tag{46}$$

where $\mathbf{r}_{\mathcal{M}}$, $\mathbf{r}_{\mathcal{C}}$, and $\mathbf{r}_{\mathcal{I}}$ are the *motion prior*, projection, and IMU residuals, respectively. $\mathbf{W}_{\mathcal{M}}$, $\mathbf{W}_{\mathcal{C}}$, and $\mathbf{W}_{\mathcal{I}}$ are the corresponding information matrices (inverse of covariance). $\mathcal{C}_t$ and $\mathcal{I}_t$ refer to the visual and inertial observations obtained during the optimization period.

### B. Ranging-Based ME

Next, we demonstrate batch optimization of range measurements with GP-based CTME. Specifically, we aim to estimate the following states:

$$\hat{\mathcal{T}} \triangleq \left(\hat{\mathcal{X}}_1, \ldots \hat{\mathcal{X}}_m\right) \tag{47}$$

with support state $\hat{\mathcal{X}}_m$ defined similarly to (44) and (45). The problem formulation for this scenario becomes minimizing the following cost function:

$$f(\hat{\mathcal{T}}) = \sum_{(\hat{\mathcal{X}}_k, \hat{\mathcal{X}}_{k+1})} \|\mathbf{r}_{\mathcal{M}}\|^2_{\mathbf{W}_{\mathcal{M}}} + \sum_{\mathcal{U}_t} \|\mathbf{r}_{\mathcal{U}}\|^2_{\mathbf{W}_{\mathcal{U}}} \tag{48}$$

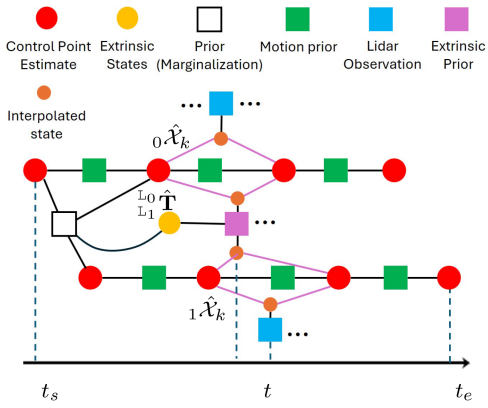



Fig. 3. Factor graph of the GP-based multi-LiDAR coupled-ME scheme. We have two trajectories corresponding to two LiDARs. Each square represents one factor, and the lines indicate the coupled states. The extrinsic prior factors reinforce the a priori.

where $\mathbf{r}_{\mathcal{M}}$ and $\mathbf{r}_{\mathcal{U}}$ denote the motion prior and range factors; $\mathbf{W}_{\mathcal{M}}$ and $\mathbf{W}_{\mathcal{U}}$ are the corresponding information matrices. $\mathcal{U}_t$ denotes a range observation within the sliding window.

### C. Multi-LiDAR Coupled-ME

In the previous scenarios, only one trajectory was represented by the GP. Inspired by the work of Peršić et al. [34], we also demonstrate a GP-based multitrajectory optimization scheme. Different from [34], which performed batch optimization for sensors with overlapping field of view (FOV), in this article, we shall investigate a MAP estimation scheme of two LiDARs that do not have any overlapping FOV. Moreover, while motion distortion is often considered undesirable in DT ME schemes, in this example, we would also like to demonstrate that CTME with GP representation can directly extract the motion in the LiDAR distortion *without relying on IMU* [41], [47]. Specifically, we define the following state estimates on a sliding window:

$$\hat{\mathcal{T}} \triangleq \left(\ldots {}_0\hat{\mathcal{X}}_m, \ldots {}_1\hat{\mathcal{X}}_m, \ldots, {}^{\mathtt{L}_0}_{\mathtt{L}_1}\hat{\mathbf{T}}\right) \tag{49}$$

$${}_i\hat{\mathcal{X}}_m = ({}_i\hat{\mathbf{R}}_m, {}_i\hat{\boldsymbol{\omega}}_m, {}_i\hat{\boldsymbol{\alpha}}_m, {}_i\hat{\mathbf{p}}_m, {}_i\hat{\mathbf{v}}_m, {}_i\hat{\mathbf{a}}_m) \tag{50}$$

where ${}_i\hat{\mathcal{X}}_m$ are the support states on the sliding window of trajectory $i$ and ${}^{\mathtt{L}_0}_{\mathtt{L}_i}\hat{\mathbf{T}} \triangleq ({}^0_i\mathbf{R}, {}^0_i\mathbf{p}) \in \mathrm{SE}(3)$ are the transformations between LiDAR 0 and LiDAR $i$. The state estimates can be optimized in the following via the cost function

$$f(\hat{\mathcal{T}}) = \|\mathbf{r}_{\mathcal{P}}\|^2_{\boldsymbol{W}_{\mathcal{P}}} + \sum_{({}_i\hat{\mathcal{X}}_k, {}_i\hat{\mathcal{X}}_{k+1})} \|\mathbf{r}_{\mathcal{M}}\|^2_{\boldsymbol{W}_{\mathcal{M}}} + \sum_{\mathcal{L}_t} \|\mathbf{r}_{\mathcal{L}}\|^2_{\boldsymbol{W}_{\mathcal{L}}} + \sum_{\tau} \|\mathbf{r}_{\mathcal{E}}\|^2_{\boldsymbol{W}_{\mathcal{E}}} \tag{51}$$

where $\mathbf{r}_{\mathcal{P}}$ and $\mathbf{r}_{\mathcal{M}}$ are, respectively, the prior from marginalization and the motion prior, $\mathbf{r}_{\mathcal{L}}$ is the residual of LiDAR-based point-to-plane factors, and $\mathbf{r}_{\mathcal{E}}$ is the residual of *extrinsic prior factor*. $\boldsymbol{W}_{\mathcal{P}}$, $\boldsymbol{W}_{\mathcal{M}}$, $\boldsymbol{W}_{\mathcal{L}}$, and $\boldsymbol{W}_{\mathcal{E}}$ are correspondingly their information matrices. Fig. 3 illustrates the factor graph based on this cost function with two LiDARs.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

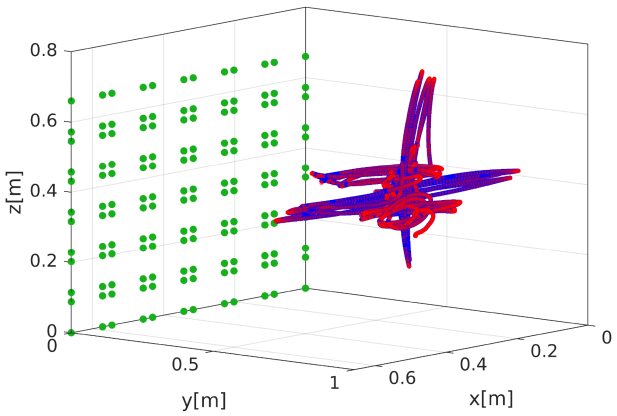



Fig. 4. Illustration of the estimated trajectory (blue) and ground truth (red) in the visual–inertial calibration, given known landmark positions depicted as green dots.

TABLE II
CALIBRATION DIFFERENCES BETWEEN GPTR AND KALIBR

| ${}^{I}_{C_0}\mathbf{R}$ [rad] | ${}^{I}_{C_1}\mathbf{R}$ [rad] | ${}^{I}_{C_0}\mathbf{p}$ [m] | ${}^{I}_{C_1}\mathbf{p}$ [m] | $\mathbf{g}$ [m/s$^2$] |
|---|---|---|---|---|
| $6.5 \times 10^{-4}$ | $2.7 \times 10^{-4}$ | 0.0012 | 0.0023 | 0.0303 |

## VII. EXPERIMENTAL VALIDATION

### A. Visual–Inertial ME

We first investigate the proposed CTME scheme in a VI optimization scheme based on a calibration problem using the real-world dataset in [50] and [51]. A stereo–visual–inertial sensor suite is assembled and captures images at 20 Hz and inertial data at 200 Hz. A checkerboard is set up w.r.t. the world frame, and the ground-truth trajectory during calibration was recorded at about 100 Hz using a motion capture system. We use the double sphere camera model [52] in this experiment. Fig. 4 visualizes the visual landmarks, ground truth, and one trajectory estimate in our experiments.

As a comparison, we incorporate the state-of-the-art tool Kalibr [9] that adopts a B-spline-based approach for VI calibration and trajectory estimation. For comparing the calibration results, we use L2 norms $\|\mathrm{Log}(\mathbf{R}_{\mathrm{Kalibr}}^{\top}\mathbf{R}_{\mathrm{GPTR}})\|$ and $\|\mathbf{p}_{\mathrm{Kalibr}} - \mathbf{p}_{\mathrm{GPTR}}\|$ for rotation and translation components, respectively. As shown in Table II, the proposed scheme delivers comparable calibration results. In addition, the proposed approach delivers an absolute position error (APE), also known as root mean square error (RMSE) of $7.7 \times 10^{-4}$m for trajectory estimation, which outperforms Kalibr with an RMSE of 1.3 m $\times\ 10^{-3}$ m.

Although the data were captured at one velocity profile, we can still emulate motion at various higher velocities. Specifically, suppose that $\breve{\boldsymbol{\omega}}_t$ and $\breve{\boldsymbol{\alpha}}_t$ are the IMU angular velocity and acceleration, with $t$ as the timestamp of the data recorded. By scaling the time stamp and the IMU data by a factor $\lambda$, i.e.,

$$t' = t/\lambda,\ \breve{\boldsymbol{\omega}}_{t'} = \lambda\breve{\boldsymbol{\omega}}_t,\ \breve{\boldsymbol{\alpha}}_{t'} = \lambda^2\breve{\boldsymbol{\alpha}}_t. \tag{52}$$

The new data with timestamp $t'$ are equivalent to being captured at a velocity that is $\lambda$ times faster.

The RMSEs of different models across the time scales are presented in Fig. 5. We can see that AP and CF models perform similarly at low speed. However, at higher speeds, the AP model starts picking up more errors. This effect can be viewed more clearly for the $\mathrm{SE}(3)$ pose representation.

The ratio of solve time between CF and AP models is presented in Fig. 6. Interestingly, we note that there are several experiments where the solve time of the CF model is lower than the AP model for the $\mathrm{SO}(3) \times \mathbb{R}^3$ representation, leading to a lower mean. We found that this is because the Ceres optimizer converges with fewer iterations when using the CF kinematic model.

*Key finding for VI ME:* In summary, for VI optimization, the CF model has similar accuracy at low speed and outperforms AP at higher velocities. It also shows that the $\mathrm{SO}(3) \times \mathbb{R}^3$+CF model can require fewer iterations to converge, thus improving the efficiency.

### B. Ranging-Based ME

To further examine the importance of pose representation for the ME task, we conduct a range-based experiment with two types of ground-truth motions as follows.

*1) Split Trajectory:* We generate a so-called *split trajectory* with ground truth as follows:

$$\mathbf{R}_t = \mathrm{Exp}\left(\boldsymbol{\theta}_t\right),\ \boldsymbol{\theta}_t = \begin{bmatrix} \pi/2\cos(\Omega t + 57) \\ \pi/2\sin(\Omega t + 57) \\ \pi\sqrt{3/2}\sin(\Omega t/3 + 43) \end{bmatrix}$$

$$\mathbf{p}_t = \begin{bmatrix} 5\sin(0.45t + 43) \\ 5\cos(0.45t + 43) \\ 5\cos(0.15t + 57) \end{bmatrix} \tag{53}$$

where $\Omega$ is varied in different experiments. This trajectory is illustrated in Fig. 8(a).

We assume that two UWB tags are offset from the body center by ${}^{B}\mathbf{x} \in \{(-0.2, 0, 0), (0.2, 0, 0)\}$. At a 50-ms interval, each UWB tag ranges to four anchors whose coordinates are $(10.0, 10.0, 0.5)$, $(-10.0, 10.0, 2.5)$, $(-10.0, -10.0, 0.5)$, and $(10.0, -10.0, 2.5)$. The measurement is corrupted by a zero-mean Gaussian noise with $0.05^2$m$^2$ variance.

We vary the frequency $\Omega$ of the trajectory ground truth and the MPI $\Delta t$ of the trajectory estimate throughout the experiments. For each variation, we initialize a 20-s-long trajectory estimate with the true pose corrupted by Gaussian noise with a $0.2^2$-rad$^2$ variance for rotation state and $0.5^2$m$^2$ variance for position state. All first and second derivatives are initialized as zeros. We then optimize the cost function with UWB and motion prior factors by the Ceres solver, with the maximum number of iterations set at 50. The RMSEs of all experiments are shown in Fig. 7(a), which is discussed below.

From Fig. 7(a), we find that at $\Delta t = 0.1$ s, the AP model starts picking up more error than CF at higher $\Omega$. We notice that the difference between AP and CF is less pronounced in $\mathrm{SO}(3) \times \mathbb{R}^3$+X setting than the $\mathrm{SE}(3)$+X setting. Notice that the RMSEs of the $\mathrm{SE}(3)$+X models also exceed 0.2 m very quickly, while $\mathrm{SO}(3) \times \mathbb{R}^3$+X models can keep the error below 0.2 m as $\Omega$ increases. These results are the evidence for the benefit of choosing the proper pose representation, as well as the CF model over the approximation model.

*2) Nonsplit Trajectory:* To confirm the benefits of choosing the appropriate pose representation with regard to the type of motion, we generate another *nonsplit trajectory* as

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

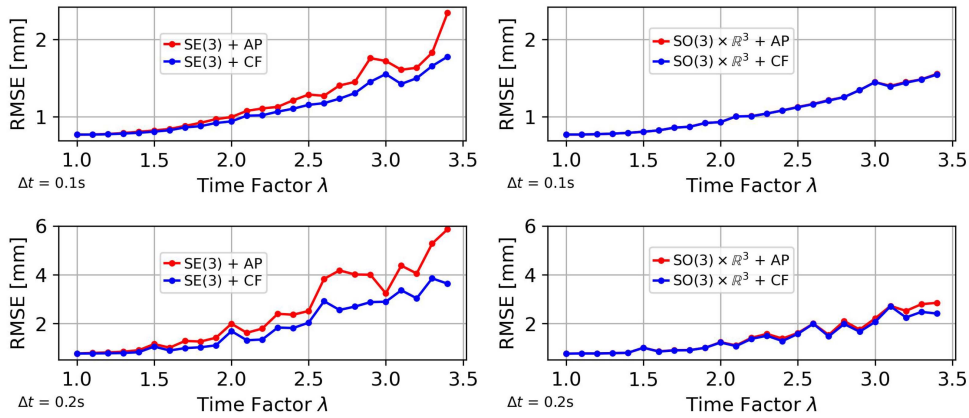



Fig. 5. RMSE of VI experiments with 0.1 s (top) and 0.2 s (bottom) MPIs under various time factors. In the legends, $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ refer to the chosen pose representations of the trajectory estimate, while AP and CF, respectively, refer to our implementation by GPTR with the AP kinematics model and the CF model. A STEAM-based implementation would be equivalent to the $\mathrm{SE}(3)$+AP settings. At low speed, the CF and AP models have similar accuracy. However, at higher speed or sparser $\Delta t$, the AP model starts gaining higher error compared to the CF model. This divergence of performance between CF and AP can be viewed more easily for the $\mathrm{SE}(3)$ (top left and bottom left) than the $\mathrm{SO}(3) \times \mathbb{R}^3$.

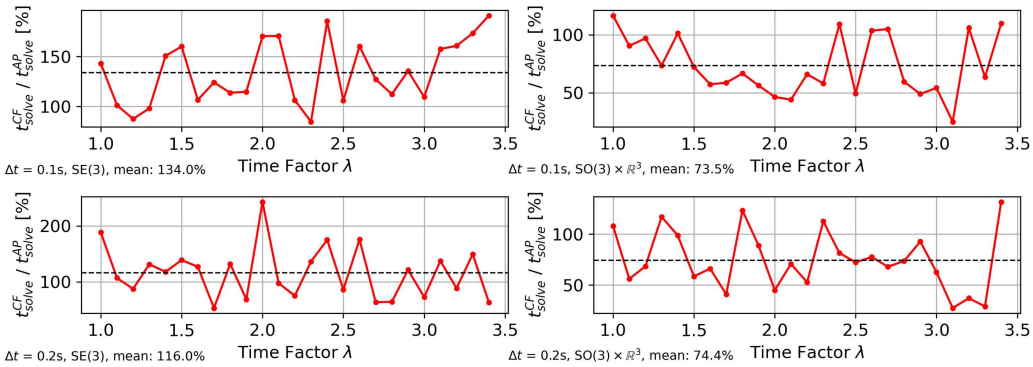



Fig. 6. Ratio of solve time between the CF and AP models corresponding to the VI experiments in Fig. 5. Notice that there are several experiments where the CF model has a much lower solve time. This is because the Ceres solver finds that the problem has converged after fewer iterations.

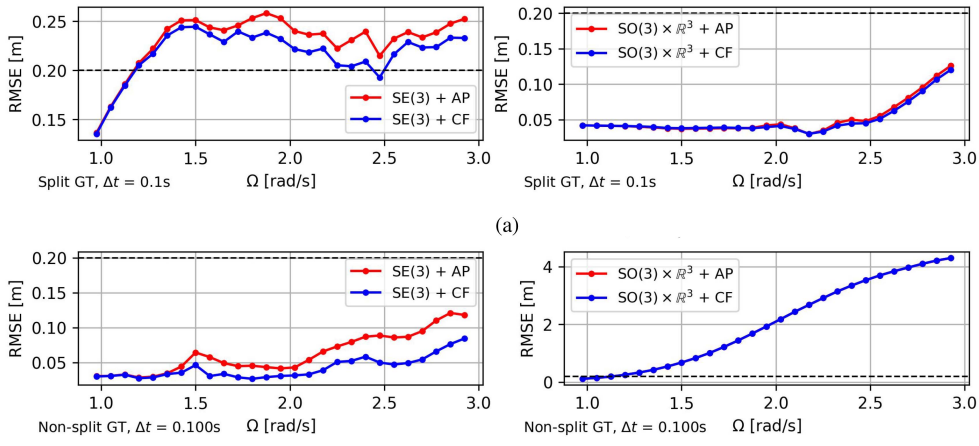
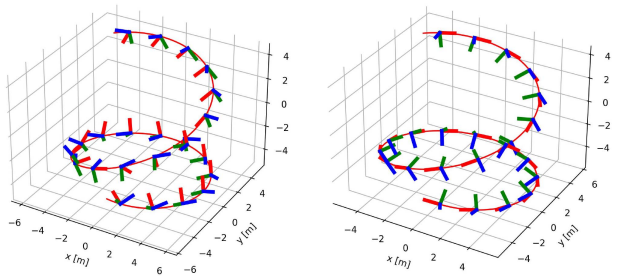



Fig. 7. RMSE of the range-only optimization experiments with different values of frequency parameter $\Omega$ . In the legends, $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ refer to the pose representation used, while AP and CF refer to the AP and CF kinematic models, respectively. In Fig. 7(a), with the split ground-truth trajectory from (53), the $\mathrm{SE}(3)$+X models (left plots) start having an RMSE above 0.2 m quickly with larger $\Omega$, while the $\mathrm{SO}(3) \times \mathbb{R}^3$+X representation (right plots) maintains an RMSE below 0.2 m. On the contrary, in Fig. 7(b), the $\mathrm{SO}(3) \times \mathbb{R}^3$ pose representation (left plots) starts losing track of the true trajectory quickly when $\Omega$ increases, while the $\mathrm{SE}(3)$ representation maintains smaller RMSE in all experiments. This demonstrates the importance of choosing the appropriate pose representation. (a) Experiments with split ground-truth trajectory (53). (b) Experiments with nonsplit ground-truth trajectory (54).

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 8. Ground-truth trajectories for experiments with UWB-only experiments in Section VII-B. (a) Split trajectory. (b) Nonsplit trajectory.

follows:

$$\mathbf{p}_t = \begin{bmatrix} 5\sin(\Omega t + 43) \\ 5\cos(\Omega t + 43) \\ 5\cos(\Omega t/3 + 57) \end{bmatrix}, \ \mathbf{R}_t = [\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z]$$

$$\mathbf{e}_x = \dot{\mathbf{p}}_t / \|\dot{\mathbf{p}}_t\|, \ \mathbf{e}_z = (\mathbf{p}_t / \|\mathbf{p}_t\|)^\wedge \mathbf{e}_x, \ \mathbf{e}_y = \mathbf{e}_z^\wedge \mathbf{e}_x. \quad (54)$$

This trajectory is visualized in Fig. 8(b). It has the same translational component as Fig. 8(a); however, the $x$-axis of the body frame is always tangent to the trajectory. This trajectory resembles that of a fixed-wing aircraft or a wheeled vehicle. All of the parameters are kept the same as in the experiments with the split trajectory, except that we flip the choice of the trajectory representation from $\mathrm{SO}(3) \times \mathbb{R}^3$ to $\mathrm{SE}(3)$. The results of these experiments are reported in Fig. 7(b).

As can be seen in Fig. 7(b), on this new trajectory, the $\mathrm{SE}(3)$ representation clearly has an advantage over $\mathrm{SO}(3) \times \mathbb{R}^3$, as the $\mathrm{SO}(3) \times \mathbb{R}^3$ RMSE quickly increases when $\Omega$ increases. Similar to the previous case, we also find that the AP model starts getting higher error than the CF model at larger $\Omega$. This again demonstrates the benefit of the CF kinematic model for trajectories with higher velocities. Moreover, although it is not obvious, this also demonstrates the flexibility of our framework, as all experiments are carried out in one single program with the same UWB factor class, and the only difference is the settings on the trajectory object.

Fig. 9 presents the ratio of the solving time for the experiments on the trajectory (53) using different methods. We can see that on average, the CF models incur a small increase in computation against the AP model for the $\mathrm{SO}(3) \times \mathbb{R}^3$ pose representation, while the $\mathrm{SE}(3)$ representation incurs approximately 90% more computation time.

*Key findings for range-based ME:* In summary, the CF kinematic model consistently outperforms AP under higher motion profiles, especially when the pose representation aligns with the trajectory type. These results highlight the effectiveness and flexibility of the proposed framework in ranging-based ME.

### C. Lidar-Based CTME

*1) Batch Optimization:* We first conduct several experiments similar to Section VII-B, where the UWB factors are replaced with point-to-plane factors. Instead of ranging to specific anchors, we simulate the LiDAR observations by finding the intersection of several LiDAR rays with the surrounding walls of a 12 m × 12 m × 8 m chamber. The rate of ray tracing is 300 Hz. This optimization can be found at the core of bundle-adjustment schemes for LiDAR-based mapping [53], [54]. The results are shown in Fig. 10(a) and (b).

From Fig. 10(a), we find that the CF and AP models still stay close, even when $\Omega$ is increased to higher values. When increasing an MPI to 0.3 s, we can start noticing a difference in $\mathrm{SO}(3) \times \mathbb{R}^3$+X models with the split ground truth.

For the split experiment, the $\mathrm{SE}(3)$+X models still diverge quickly (thus, it is not meaningful to compare the performance between CF and AP, as the errors have diverged to some random values). Conversely, for the nonsplit ground truth, the $\mathrm{SO}(3) \times \mathbb{R}^3$+X models diverge quickly, while $\mathrm{SE}(3)$+X models can maintain estimation capacity for longer. Based on this experiment, we conclude that the CF may not have a clear advantage when applying with point-to-plane factors. It remains consistent with previous experiments that there is a benefit in choosing the appropriate pose representation for different motion types.

*Key finding for LiDAR-based CTME:* In this experiment, the LiDAR-based CTME experiments show no clear performance difference between the CF and AP models in typical operations. We posit that this close performance is due to the higher density of the LiDAR factors in the batch optimisation problems (300 Hz compared to 20 Hz in the ranging scenario). As the number of factors on each MPI increases, the reduced ratio of true trajectory variation over the observation error makes the comparative expressiveness of the two models less significant. Only when using a sparser representation (increased MPI), then the CF model can be shown to better capture the long-term trend of the motion.

In the next parts, we will focus more on practical applications of GP-based CTME and compare its performance against existing methods of lower orders.

*2) Real-World Handheld Setup:* In this section, we conduct an experiment of the multi-LiDAR coupled-motion estimation (MLCME) scheme on a handheld setup with two LiDARs. Three state-of-the-art LiDAR-only LiDAR-odometry (LO) methods are selected for comparison: I2EKF-LO [55], Traj-LO [47], and continuous-time estimation for multi-lidar odometry (CTE-MLO) [56]. In the frontend, all methods use a similar point-to-plane factor with slightly different feature association strategies: I2EKF-LO, Traj-LO, and GPTR use K-nearest neighbors (KNN) search, while CTE-MLO uses a voxel map. In the backend, we disable the mapping process and only use a fixed prior map for all methods. The similarities in both frontend and backend allow us to focus the comparison on the trajectory representation and the kinematic model. I2EKF uses a second-order WNOJ; Traj-LO is a constant velocity model with $\mathrm{SE}(3)$ pose representation; CTE-MLO uses a second-order $\mathrm{SO}(3)$ GP and a third-order GP in the translational part.

We first test the multi-LiDAR calibration scheme on a real-world setup, as shown in Fig. 12, which consists of two Livox-Mid360 LiDARs. The two sensors' internal clocks are synchronized by a common trigger from the onboard computer's input-output (IO) pin. We use the datasheet illustrations, as well as the hardware computer-aided design (CAD) model, to deduce the ground truth for the extrinsics. The ground-truth extrinsics are directly used to enable multi-LiDAR capability of CTE-MLO (and fixed for the whole experiments), while the extrinsics in the GPTR experiments are initialized with an identity transform. For Traj-LO and GPTR, the sliding window progresses at 0.02204 s interval, which is also the MPI of

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

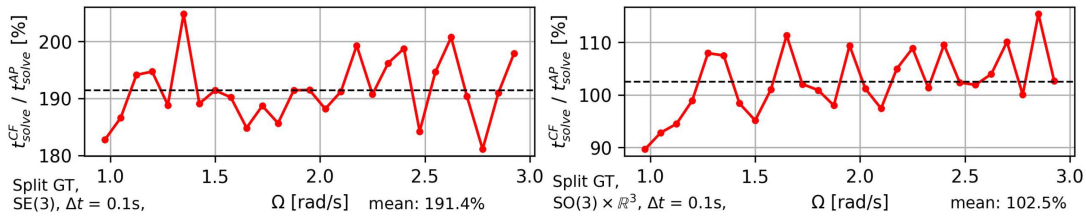



Fig. 9. Ratio of solving time by the CF kinematics model and the AP model in the ranging-based optimization problem, using $\mathrm{SO}(3) \times \mathbb{R}^3$ (left) and $\mathrm{SE}(3)$ (right) representations.

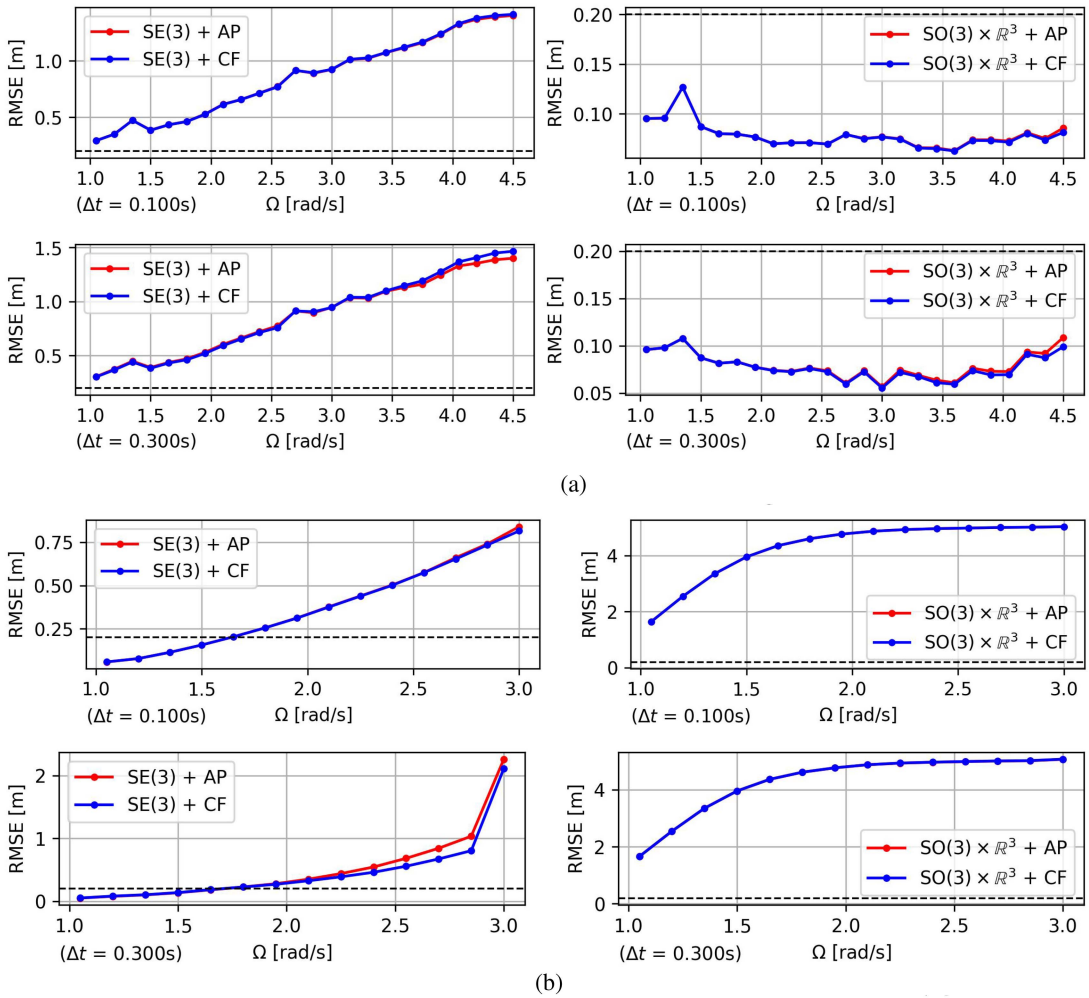



Fig. 10. RMSE of the LiDAR-based batch optimization experiments. (a) RMSE of the LiDAR-based optimization at different MPIs under the split trajectory defined in (53). (b) RMSE of the LiDAR-based optimization at different MPIs under the nonsplit trajectory defined in (54).

GPTR. We also run GPTR with fixed ground-truth extrinsics as CTE-MLO, to compare with the dynamic extrinsic (DX) estimation scheme. The trajectory ground truth is obtained from a motion capture system, with careful compensation for the reflective markers and the LiDAR's frame of reference.

Fig. 13 shows the trajectory estimates of the LiDARs and the ground truth. It can be seen that in this experiment, the motion is quite aggressive, where the setup is swung around in the experiment area. We calculate the APE of the trajectory estimates from I2EKF, Traj-LO, CTE-MLO and our method, and report it in Table III. The table shows that the MLCME scheme using the $\mathrm{SO}(3) \times \mathbb{R}^3$ model receives the highest accuracy, followed by the $\mathrm{SE}(3)$. We also notice the accuracy of CTE-MLO is close to GPTR's, thanks to its use of multiple LiDARs, which enjoys more robustness, thanks to extended FOV. In contrast, I2EKF and Traj-LO, being single-LiDAR methods, have higher errors on each LiDAR's trajectory. In Fig. 14, we can also see that the extrinsic parameters converge to some constant values, which are within 1.5 cm and 2° of our assumed ground truth. We notice that for LiDAR 0, the accuracy of DX method is higher than fixed extrinsics (FX), and vice versa for LiDAR 1. However, the benefit/disadvantage between DX and FX is not quite obvious, as the differences are quite small. This can be more conclusively established in the next experiment in Section VII-C3. In Fig. 11,

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

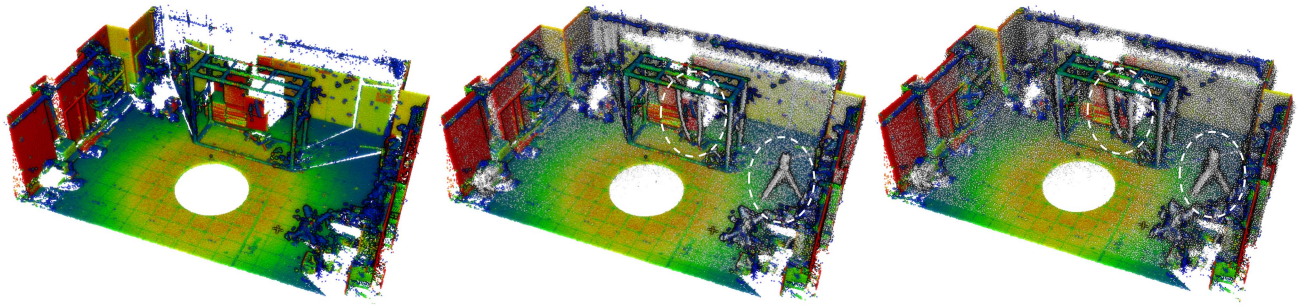


Fig. 11. Left: The prior map. Middle: The LiDAR point clouds, undistorted and overlaid on the prior map. Right: The distorted raw point cloud over the prior map.

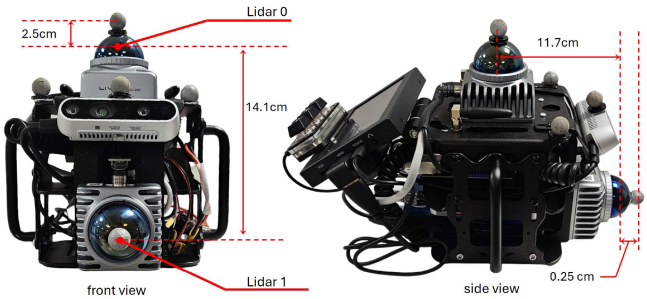



Fig. 12. Two-LiDAR sensor suite. Based on the Mid360 Datasheet, we assume that the origin of the LiDAR's coordinate system coincides with the center of the half-spheres.

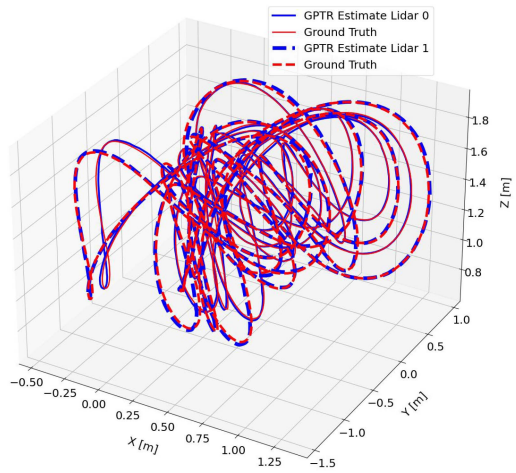



Fig. 13. Trajectory estimate for each LiDAR by the MLCME scheme versus the ground truth.

we also show the pointclouds undistorted by the CT trajectory overlaid on top of the prior map, compared to that without distortion.

*3) Synthetic Data:* In generating the synthetic dataset, we assume that two LiDARs, 0 and 1, are mounted on a vehicle (see Fig. 15) and their extrinsics are set as a user-defined parameter. The trajectory of the vehicle is predetermined by a continuous function where $\mathbf{p}_t = (2\sin(\Omega t), 2\sin(\Omega t)\cos(\Omega t), 0.75)$ and $\mathbf{R}_t$ is the same as in (54). The parameter $\Omega$ is varied in different sequences. We then sample the vehicle's pose every 2e-5s, convert it to the LiDAR's pose, perform ray-tracing to the walls of a 6 m $\times$ 6 m room to obtain the distance, add a Gaussian noise of 5 cm variance to the range, and finally calculate the 3-D point. The coordinate of LiDAR 0 and LiDAR 1 in the robot body frame expressed in a yaw–pitch–roll–x–y–z tuple is set as

$$ {}^{\mathrm{B}}_{\mathrm{L}_0}\mathbf{T} = (45^\circ, 0, 0, 0, 0, 0) $$

TABLE III
COMPARING THE RMSE OF TRAJECTORY ESTIMATE BY THE MLCME SCHEME IN GPTR AND OTHER METHODS WITH THE HANDHELD SETUP DATA

| S# | L# | I$^2$EKF | Traj | CTEM | GPTR | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | SE(3) | | SO(3) $\times$ $\mathbb{R}^3$ | |
| | | | | | DX | FX | DX | FX |
| 7 | 0 | 0.043 | 0.0476 | 0.027 | 0.0236 | 0.0257 | **0.0232** | 0.0241 |
| | 1 | 0.0417 | 0.6793 | 0.0442 | 0.0296 | 0.0258 | 0.0283 | **0.0256** |
| 8 | 0 | 0.042 | 0.0647 | 0.0278 | 0.0229 | 0.0259 | **0.0223** | 0.0246 |
| | 1 | 0.0418 | 0.043 | 0.1319 | 0.032 | 0.0274 | 0.0313 | **0.0271** |
| 9 | 0 | 0.0554 | 0.0477 | 0.0422 | 0.0395 | 0.0411 | **0.039** | 0.0406 |
| | 1 | 0.0567 | x | 0.0568 | 0.0452 | 0.0436 | 0.0444 | **0.0433** |

Note: All values are in [m] unit. **S#** and **L#** are sequence and LiDAR indices. **Traj** and **CTEM** are shorthand for Traj-LO [47] and CTE-MLO [56]. DX and FX are abbreviations for when extrinsics estimation is set to "dynamic" or "fixed" during optimization. The methods in GPTR use the closed-form kinematic model. The best result in each row is in **bold**, while the second best is underlined.

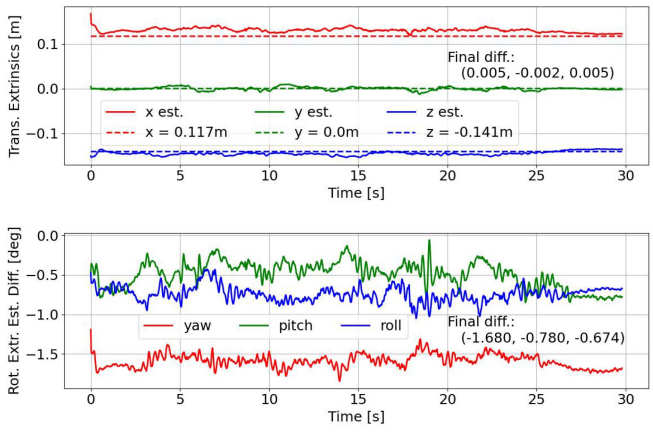



Fig. 14. Extrinsic estimation result between LiDAR 0 and LiDAR 1. The upper plot shows the translation extrinsic values, while the lower plot shows the rotational difference between the estimate and the ground truth.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

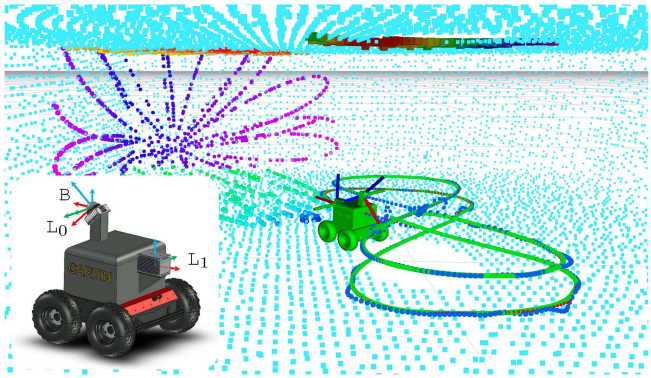



Fig. 15. Simulated setup with two LiDARs mounted on top of a ground robot. The ground robot moves in a room with six flat sides. The two LiDARs have no overlap. The GPTR trajectory and I2EKF-LO estimates are marked by green and blue, respectively, while ground-truth trajectories are in red.

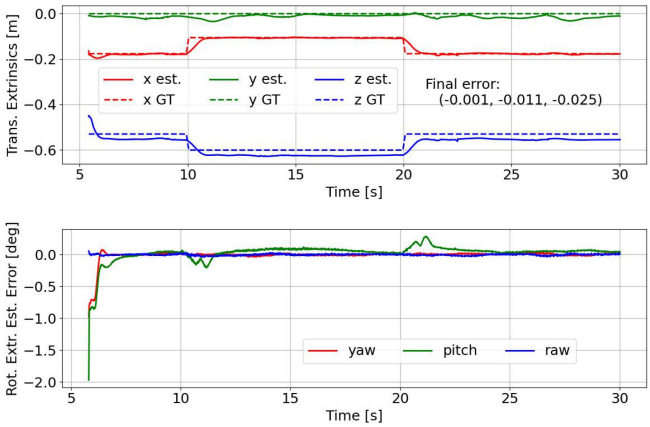



Fig. 16. Estimation error of the extrinsic parameters between LiDAR 0 and LiDAR 1 in the synthetic dataset.

$$ {}^{\mathtt{B}}_{\mathtt{L}_1}\mathbf{T} = \begin{cases} (180^\circ, 0, 0, -0.5, 0, -0.25), & \text{if } t \notin (10, 20) \\ (180^\circ, 0, 0, -0.5, 0, -0.35), & \text{otherwise.} \end{cases} $$

Hence, the two LiDARs' FOV do not overlap. From ${}^{\mathtt{B}}_{\mathtt{L}_0}\mathbf{T}$ and ${}^{\mathtt{B}}_{\mathtt{L}_1}\mathbf{T}$, we can calculate the transform between $\mathtt{L}_0$ and $\mathtt{L}_1$ as

$$ {}^{\mathtt{L}_0}_{\mathtt{L}_1}\mathbf{T} = \begin{cases} (180^\circ, 45^\circ, 0^\circ, \frac{-1}{4\sqrt{2}}, 0, \frac{-3}{4\sqrt{2}}), & \text{if } t \notin (10, 20) \\ (180^\circ, 45^\circ, 0^\circ, \frac{-3}{20\sqrt{2}}, 0, \frac{-17}{20\sqrt{2}}), & \text{if } t \in (10, 20) \end{cases} $$

which is the ground truth for the MLCME scheme. The change of extrinsic mimics a scenario where the sensor mechanical holder may slip during operation. The LiDAR initial poses are set with 10 cm translation and 5° rotation errors. The MPI is chosen to be 0.04357 s. Fig. 15 visualizes the setup of our experiments with synthetic data.

Table IV shows that GPTR achieves the lowest RMSE in the trajectory estimates of both LiDARs. In Fig. 16, we can see that the MLCME scheme can estimate the LiDAR extrinsic dynamically: the extrinsic parameters converge quickly within the first few seconds, and at $t = 10$, when there is a significant change in the extrinsic parameters, the MLCME scheme can quickly correct its estimate. Moreover, we can also see that while other methods have their RMSE increased significantly as $\Omega$ increases, the increase of GPTR method is much milder, showing that the third-order model has a higher estimation capacity than the models of lower order.

TABLE IV
COMPARING THE RMSE OF TRAJECTORY ESTIMATE BY GPTR'S MLCME SCHEME AND OTHER METHODS WITH THE SYNTHETIC DATA AT DIFFERENT VALUES OF $\Omega$

| S# | L# | I$^2$EKF | Traj | CTEM | GPTR | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | SE(3) | | SO(3) × $\mathbb{R}^3$ | |
| | | | | | DX | FX | DX | FX |
| 0.25 | 0 | 0.0571 | 0.0635 | 0.0363 | 0.0339 | 0.0259 | 0.0338 | **0.0259** |
| | 1 | 0.0495 | 0.0585 | 0.0397 | 0.039 | 0.0325 | 0.0391 | **0.0324** |
| 0.35 | 0 | 0.0576 | 0.0633 | 0.0359 | 0.0338 | **0.0274** | 0.0335 | 0.0275 |
| | 1 | 0.0506 | 0.0586 | 0.0419 | 0.0407 | 0.0354 | 0.0409 | **0.0354** |
| 0.45 | 0 | 0.058 | 0.0632 | 0.0378 | 0.0342 | **0.0287** | 0.0339 | 0.0288 |
| | 1 | 0.0531 | 0.0593 | 0.0456 | 0.0413 | **0.0382** | 0.0415 | 0.0383 |
| 0.55 | 0 | 0.0583 | 0.0626 | 0.038 | 0.0349 | **0.0304** | 0.0342 | 0.0307 |
| | 1 | 0.0564 | 0.0624 | 0.0469 | 0.0428 | **0.0412** | 0.0432 | 0.0413 |
| 0.65 | 0 | 0.0595 | 0.063 | 0.0396 | 0.0347 | **0.032** | 0.0342 | 0.0326 |
| | 1 | 0.0621 | 0.0657 | 0.05 | **0.0434** | 0.0437 | 0.0442 | 0.044 |
| 0.75 | 0 | 0.0617 | 0.0623 | 0.0426 | 0.0357 | **0.0326** | 0.035 | 0.0331 |
| | 1 | 0.0693 | 0.0715 | 0.053 | **0.0449** | 0.0466 | 0.0461 | 0.0472 |
| 0.85 | 0 | 0.0632 | 0.0629 | 0.0456 | 0.0365 | **0.0355** | 0.0357 | 0.037 |
| | 1 | 0.0756 | 0.08 | 0.0556 | **0.046** | 0.0487 | 0.0472 | 0.0493 |
| 0.95 | 0 | 0.0671 | 0.0641 | 0.0561 | 0.0368 | 0.0376 | **0.0357** | 0.0396 |
| | 1 | 0.0856 | 0.0905 | 0.0669 | **0.0472** | 0.0517 | 0.0491 | 0.0528 |

Note: All values are in [m] unit. **S#** and **L#** are sequence and LiDAR indices. **Traj** and **CTEM** are shorthand for Traj-LO [47] and CTE-MLO [56]. DX and FX are abbreviations for when extrinsics estimation is set to "dynamic" or "fixed" during optimization. The methods in GPTR use the closed-form kinematic model. The best result in each row is in **bold**, while the second best is underlined.

In Table IV, there is a shift in accuracy from slower sequences to faster sequences; however, it is not very decisive to conclude on the comparative advantage of the DX and FX schemes. Hence, we calculate the RMSE within only the interval where $t \in (9, 21)$ for GPTR experiments, which contains the $(10, 20)$ period where the extrinsics change substantially. The results are shown in Table V. It is clear from the results that the DX scheme allows the trajectory estimate to adapt more flexibly to the extrinsic changes.

*Remark 5:* Our example above demonstrates the efficacy of GPTR in a multitrajectory calibration scenario; thus, a map of the room was assumed available (in practice, it can be obtained by using a 3-D scanner or by stitching together some static LiDAR scans). Without a prior map, one can adopt a batch optimization scheme with iterative feature matching [54], [57], [58]. The example can also be modified to become a full-fledged LiDAR odometry and mapping (LOAM) system with online calibration by integrating some incremental mapping technique, for example, ikd-Tree [59], VoxelMap [60], and UFOMap [61]. To the best of the authors' knowledge, few works have addressed the online calibration of multi-LiDAR systems, and most works often assume a constant LiDAR extrinsic. In the rare case of M-LIOM [62] that does consider online calibration, the method requires overlap between sensors for it to work.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARING THE RMSE OF TRAJECTORY ESTIMATE BY GPTR'S MLCME SCHEMES IN THE TRANSIENCE PERIOD $t \in (9, 21)$

| S# | L# | GPTR | | | |
|---|---|---|---|---|---|
| | | $\mathrm{SE}(3)$ | | $\mathrm{SO}(3) \times \mathbb{R}^3$ | |
| | | DX | FX | DX | FX |
| 0.25 | 0 | **0.0344** | 0.0357 | **0.0344** | 0.0357 |
| | 1 | **0.0401** | 0.0565 | **0.0401** | 0.0564 |
| 0.35 | 0 | **0.0352** | 0.0362 | **0.0349** | 0.0362 |
| | 1 | **0.0408** | 0.0561 | **0.0409** | 0.0560 |
| 0.45 | 0 | **0.0343** | 0.0354 | **0.0342** | 0.0356 |
| | 1 | **0.0411** | 0.0560 | **0.0414** | 0.0561 |
| 0.55 | 0 | **0.0346** | 0.0357 | **0.0340** | 0.0362 |
| | 1 | **0.0423** | 0.0566 | **0.0429** | 0.0564 |
| 0.65 | 0 | **0.0347** | 0.0368 | **0.0341** | 0.0372 |
| | 1 | **0.0431** | 0.0566 | **0.0439** | 0.0566 |
| 0.75 | 0 | **0.0348** | 0.0363 | **0.0339** | 0.0366 |
| | 1 | **0.0433** | 0.0576 | **0.0442** | 0.0579 |
| 0.85 | 0 | **0.0351** | 0.0374 | **0.0334** | 0.0386 |
| | 1 | **0.0438** | 0.0569 | **0.0448** | 0.0570 |
| 0.95 | 0 | **0.0352** | 0.0387 | **0.0333** | 0.0400 |
| | 1 | **0.0448** | 0.0582 | **0.0470** | 0.0587 |

Note: All values are in [m] unit. **S#**, **L#**, DX and FX are defined similarly to Tab. IV. The methods in GPTR use the closed-form kinematic model. The better result between DX and FX under each pose representation is in **bold**.

## VIII. CONCLUSION

In this article, we have developed a CT trajectory representation framework based on third-order GP, featuring a CF kinematic model for both $\mathrm{SO}(3) \times \mathbb{R}^3$ and $\mathrm{SE}(3)$ pose representation. These kinematics models are encapsulated in a unified framework, thus making it easier for users to experiment and compare different configurations. We also include a comprehensive set of working examples of ME, leveraging LiDAR, visual, IMU, and UWB factors in both batch-optimization and sliding-window optimization schemes. The experiments have also provided crucial insights into the benefits of CF kinematic models over traditional approximate models. Based on these examples, we also hope that the community can come up with more advanced ME schemes featuring other sensing models. Our code base named GPTR has been open-sourced online,[5] and we welcome all feedback and contributions from the community to improve our project.

## APPENDIX A
## CF OF $\boldsymbol{J}_r(\boldsymbol{u})$JR, $\boldsymbol{J}_r^{-1}(\boldsymbol{u})$JRINV, $\boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v})$HTHE, AND $\boldsymbol{H}_1'(\boldsymbol{u},\boldsymbol{v})$H'THE

Let us first define the $f(\cdot)$ functions in (40)

$$f(\boldsymbol{u}) \triangleq (\boldsymbol{u}^\wedge)^2 \tag{55}$$

$$f_u(\boldsymbol{u},\boldsymbol{v}) \triangleq \frac{\partial f(\boldsymbol{u})\boldsymbol{v}}{\partial \boldsymbol{u}} = -\boldsymbol{u}^\wedge\boldsymbol{v}^\wedge - (\boldsymbol{u}^\wedge\boldsymbol{v})^\wedge \tag{56}$$

$$f_{uu}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) \triangleq \frac{\partial f_u(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{u}} = (\boldsymbol{v}^\wedge\boldsymbol{w})^\wedge - \boldsymbol{w}^\wedge\boldsymbol{v}^\wedge \tag{57}$$

$$f_{uv}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) \triangleq \frac{\partial f_u(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{v}} = \boldsymbol{u}^\wedge\boldsymbol{w}^\wedge + \boldsymbol{w}^\wedge\boldsymbol{u}^\wedge \tag{58}$$

where $\boldsymbol{u}, \boldsymbol{v},$ and $\boldsymbol{w}$ are $\mathbb{R}^3$ vectors.

For a vector $\boldsymbol{u}$, let us define the following:

$$u \triangleq \|\boldsymbol{u}\|, \ \bar{\boldsymbol{u}} \triangleq \frac{\boldsymbol{u}}{u}, \ \bar{\boldsymbol{u}}_{\boldsymbol{u}} \triangleq (\mathbf{I} - \bar{\boldsymbol{u}}\bar{\boldsymbol{u}}^\top)\frac{1}{u}. \tag{59}$$

We then define the $g(\cdot)$ functions and its derivatives

$$g_1^{(0)}(u) = g_1(u) \triangleq \frac{1-\cos u}{u^2} \tag{60}$$

$$g_1^{(1)}(u) = \frac{\sin u}{u^2} + \frac{2(\cos u - 1)}{u^3} \tag{61}$$

$$g_1^{(2)}(u) = \frac{\cos u}{u^2} - \frac{4\sin u}{u^3} + \frac{6 - 6\cos u}{u^4} \tag{62}$$

$$g_2^{(0)}(u) = g_2(u) \triangleq \frac{u - \sin u}{u^3} \tag{63}$$

$$g_2^{(1)}(u) = -\frac{2}{u^3} - \frac{\cos u}{u^3} + \frac{3\sin u}{u^4} \tag{64}$$

$$g_2^{(2)}(u) = \frac{\sin u}{u^3} + \frac{6\cos u + 6}{u^4} - \frac{12\sin u}{u^5} \tag{65}$$

$$g_3^{(0)}(u) = g_3(u) \triangleq \frac{1}{u^2} - \frac{1+\cos u}{2\sin u} \tag{66}$$

$$g_3^{(1)}(u) = \frac{-2}{u^3} + \frac{(1+\cos u)(u+\sin u)}{2u^2(\sin u)^2} \tag{67}$$

$$g_3^{(2)}(u) = \frac{6}{u^4} + \frac{\sin u}{u^3(\cos u - 1)} + \frac{u\cos u + 2\sin u + u}{2u^2\sin u(\cos u - 1)}. \tag{68}$$

In addition, we also define the following Jacobians:

$$\mathbf{J}_{\boldsymbol{u}}^{g_j^{(i)}} \triangleq g_j^{(i+1)}(u)\bar{\boldsymbol{u}}^\top. \tag{69}$$

Hence, the CF of $\boldsymbol{J}_r$ and $\boldsymbol{J}_r^{-1}$ as provided in [48] are recalled as follows:

$$\boldsymbol{J}_r(\boldsymbol{u}) = \mathbf{I} - g_1(u)\boldsymbol{u}^\wedge + g_2(u)f(\boldsymbol{u}) \tag{70}$$

$$\boldsymbol{J}_r^{-1}(\boldsymbol{u}) = \mathbf{I} + \frac{1}{2}\boldsymbol{u}^\wedge + g_3(u)f(\boldsymbol{u}). \tag{71}$$

Based on the forms of $\boldsymbol{J}_r(\boldsymbol{u})$ and $\boldsymbol{J}_r^{-1}(\boldsymbol{u})$ above, we apply the procedure described in Section IV-C to achieve the CF of $\boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v})$ and $\boldsymbol{H}_1'(\boldsymbol{u},\boldsymbol{v})$. Specifically

$$\boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v}) = \boldsymbol{v}^\wedge g_1(u) + \boldsymbol{v}^\wedge\boldsymbol{u}\mathbf{J}_{\boldsymbol{u}}^{g_1^{(0)}} + f_u(\boldsymbol{u},\boldsymbol{v})g_2(u) + f(\boldsymbol{u})\boldsymbol{v}\mathbf{J}_{\boldsymbol{u}}^{g_2^{(0)}} \tag{72}$$

$$\boldsymbol{H}_1'(\boldsymbol{u},\boldsymbol{v}) = -\frac{1}{2}\boldsymbol{v}^\wedge + f_u(\boldsymbol{u},\boldsymbol{v})g_3(u) + f(\boldsymbol{u})\boldsymbol{v}\mathbf{J}_{\boldsymbol{u}}^{g_3^{(0)}}. \tag{73}$$

[5][Online]. Available: https://github.com/brytsknguyen/gptr.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

In Appendix B, we apply the procedure in Section IV-C once more on $\boldsymbol{H}$ and $\boldsymbol{H}'$, which will be used to calculate the intrinsic Jacobians in Section C.

## Appendix B
## CF of $\boldsymbol{L}$L and $\boldsymbol{L}'$L' Mappings

$$\begin{aligned}\boldsymbol{L}_{11}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) &= \frac{\partial \boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{u}} \\ &= \boldsymbol{v}^{\wedge}\boldsymbol{w}\mathbf{J}_{\boldsymbol{u}}^{g_1^{(0)}} + \boldsymbol{v}^{\wedge}(\mathbf{J}_{\boldsymbol{u}}^{g_1^{(0)}}\boldsymbol{w}) \\ &+ \boldsymbol{v}^{\wedge}\boldsymbol{u}(\bar{\boldsymbol{u}}^{\top}\boldsymbol{w})\mathbf{J}_{\boldsymbol{u}}^{g_1^{(1)}} + \boldsymbol{v}^{\wedge}\boldsymbol{u}g_1^{(1)}(u)\boldsymbol{w}^{\top}\bar{\boldsymbol{u}}_{\boldsymbol{u}} \\ &+ f_{uu}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})g_2(u) + f_u(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}\mathbf{J}_{\boldsymbol{u}}^{g_2^{(0)}} \\ &+ f_u(\boldsymbol{u},\boldsymbol{v})(\mathbf{J}_{\boldsymbol{u}}^{g_2^{(0)}}\boldsymbol{w}) + f(\boldsymbol{u},\boldsymbol{v})g_2^{(1)}(u)\boldsymbol{w}^{\top}\bar{\boldsymbol{u}}_{\boldsymbol{u}} \\ &+ f(\boldsymbol{u})\boldsymbol{v}\bar{\boldsymbol{u}}^{\top}\boldsymbol{w}\mathbf{J}_{\boldsymbol{u}}^{g_2^{(1)}} \end{aligned} \tag{74}$$

$$\begin{aligned}\boldsymbol{L}_{12}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) &= \frac{\partial \boldsymbol{H}_1(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{v}} \\ &= -\boldsymbol{w}^{\wedge}g_1(u) - \boldsymbol{u}^{\wedge}\mathbf{J}_{\boldsymbol{u}}^{g_1^{(0)}}\boldsymbol{w} + f_{uv}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})g_2(u) \\ &+ f_v(\boldsymbol{u})\boldsymbol{v}(\mathbf{J}_{\boldsymbol{u}}^{g_2^{(0)}}\boldsymbol{w}) \end{aligned} \tag{75}$$

$$\begin{aligned}\boldsymbol{L}'_{11}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) &= \frac{\partial \boldsymbol{H}'_1(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{u}} \\ &= f_{uu}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})g_3(u) + f_u(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}\mathbf{J}_{\boldsymbol{u}}^{g_3^{(0)}} \\ &+ f_u(\boldsymbol{u},\boldsymbol{v})\mathbf{J}_{\boldsymbol{u}}^{g_3^{(0)}}\boldsymbol{w} + f(\boldsymbol{u},\boldsymbol{v})g_3^{(1)}(u)\boldsymbol{w}^{\top}\bar{\boldsymbol{u}}_{\boldsymbol{u}} \\ &+ f(\boldsymbol{u},\boldsymbol{v})\boldsymbol{w}^{\top}\bar{\boldsymbol{u}}\mathbf{J}_{\boldsymbol{u}}^{g_3^{(1)}} \end{aligned} \tag{76}$$

$$\begin{aligned}\boldsymbol{L}'_{12}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}) &= \frac{\partial \boldsymbol{H}'_1(\boldsymbol{\theta},\boldsymbol{v})\boldsymbol{w}}{\partial \boldsymbol{v}} \\ &= \frac{1}{2}\boldsymbol{w}^{\wedge} + f_{uv}(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})g_3^{(0)}(u) + f_v(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})\mathbf{J}_{\boldsymbol{u}}^{g_3^{(0)}}. \end{aligned} \tag{77}$$

## Appendix C
## Intrinsic Jacobians

### *C1* $\mathrm{SO}(3)\times\mathbb{R}^3$ *Representation*

We first detail the intrinsic Jacobians of the $\mathrm{SO}(3)$ GP. Afterward, the vector GP can be laid out easily. To simplify the notation, we denote $\mathbf{J}_{\mathcal{X}}^{\mathcal{Y}} \triangleq \frac{\partial \mathcal{Y}}{\partial \mathcal{X}}$. In Fig. 2(a), we provide a diagram of the relationships between the variables. Our goal is to find all of the Jacobians $\mathbf{J}_{\mathcal{X}}^{\mathcal{Y}}$ where $\mathcal{Y}$ belongs to L4 and $\mathcal{X}$ belongs to L1. This is done in a layer-by-layer manner.

First, we calculate the Jacobians of variables in L2 over those in L1. By using the diagram in Fig. 2(a), we can identify eleven L1-L0 connections, each can be represented by one Jacobian. Note that a connection can have multiple distinct paths. Specifically, from $\boldsymbol{\gamma}_a$ states, we have two connections by two simple paths

$$\mathbf{J}_{\boldsymbol{\omega}_a}^{\dot{\boldsymbol{\theta}}_a} = \mathbf{I},\ \mathbf{J}_{\boldsymbol{\alpha}_a}^{\ddot{\boldsymbol{\theta}}_a} = \mathbf{I}. \tag{78}$$

From $\boldsymbol{\theta}_b$, we can identify two connections

$$\mathbf{J}_{\mathbf{R}_a}^{\boldsymbol{\theta}_b} = \frac{\partial \mathrm{Log}(\mathbf{R}_a^{-1}\mathbf{R}_b)}{\partial \mathbf{R}_a} = -\boldsymbol{J}_r^{-1}(-\boldsymbol{\theta}_b) \tag{79}$$

$$\mathbf{J}_{\mathbf{R}_b}^{\boldsymbol{\theta}_b} = \frac{\partial \mathrm{Log}(\mathbf{R}_a^{-1}\mathbf{R}_b)}{\partial \mathbf{R}_b} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b). \tag{80}$$

From $\dot{\boldsymbol{\theta}}_b$, there are three connections based on three paths

$$\begin{aligned}\mathbf{J}_{\mathbf{R}_a}^{\dot{\boldsymbol{\theta}}_b} &= \frac{\partial[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b)\boldsymbol{\omega}_b]}{\partial \mathbf{R}_a} = \frac{\partial[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b)\boldsymbol{\omega}_b]}{\partial \boldsymbol{\theta}_b}\frac{\partial \boldsymbol{\theta}_b}{\partial \mathbf{R}_a} \\ &= \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\mathbf{J}_{\mathbf{R}_a}^{\boldsymbol{\theta}_b} \end{aligned} \tag{81}$$

$$\mathbf{J}_{\mathbf{R}_b}^{\dot{\boldsymbol{\theta}}_b} = \frac{\partial[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b)\boldsymbol{\omega}_b]}{\partial \mathbf{R}_b} = \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\mathbf{J}_{\mathbf{R}_b}^{\boldsymbol{\theta}_b} \tag{82}$$

$$\mathbf{J}_{\boldsymbol{\omega}_b}^{\dot{\boldsymbol{\theta}}_b} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b). \tag{83}$$

From $\ddot{\boldsymbol{\theta}}_b$, there are two paths to $\mathbf{R}_a$, hence

$$\begin{aligned}\mathbf{J}_{\mathbf{R}_a}^{\ddot{\boldsymbol{\theta}}_b} &= \frac{\partial[\boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b)\boldsymbol{\alpha}_b + \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\dot{\boldsymbol{\theta}}_b]}{\partial \mathbf{R}_a} \\ &= \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\alpha}_b)\mathbf{J}_{\mathbf{R}_a}^{\boldsymbol{\theta}_b} + \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\mathbf{J}_{\mathbf{R}_a}^{\dot{\boldsymbol{\theta}}_b} \\ &+ \boldsymbol{\mathcal{L}'}_{11}(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b,\dot{\boldsymbol{\theta}}_b)\mathbf{J}_{\mathbf{R}_a}^{\boldsymbol{\theta}_b} \end{aligned} \tag{84}$$

where $L'$ is defined in Appendix B.

Similarly, from $\ddot{\boldsymbol{\theta}}_b$ to $\mathbf{R}_b$, there are three paths, $\ddot{\boldsymbol{\theta}}_b$ to $\boldsymbol{\omega}_b$ two, and $\ddot{\boldsymbol{\theta}}_b$ to $\boldsymbol{\alpha}_b$ one, yielding three Jacobians

$$\begin{aligned}\mathbf{J}_{\mathbf{R}_b}^{\ddot{\boldsymbol{\theta}}_b} &= \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\alpha}_b)\mathbf{J}_{\mathbf{R}_b}^{\boldsymbol{\theta}_b} + \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\mathbf{J}_{\mathbf{R}_b}^{\dot{\boldsymbol{\theta}}_b}, \\ &+ \boldsymbol{\mathcal{L}'}_{11}(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b,\dot{\boldsymbol{\theta}}_b)\mathbf{J}_{\mathbf{R}_b}^{\boldsymbol{\theta}_b} \end{aligned} \tag{85}$$

$$\mathbf{J}_{\boldsymbol{\omega}_b}^{\ddot{\boldsymbol{\theta}}_b} = \boldsymbol{\mathcal{L}'}_{12}(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b,\dot{\boldsymbol{\theta}}_b) + \boldsymbol{H}'_1(\boldsymbol{\theta}_b,\boldsymbol{\omega}_b)\mathbf{J}_{\boldsymbol{\omega}_b}^{\dot{\boldsymbol{\theta}}_b} \tag{86}$$

$$\mathbf{J}_{\boldsymbol{\alpha}_b}^{\ddot{\boldsymbol{\theta}}_b} = \boldsymbol{J}_r^{-1}(\boldsymbol{\theta}_b). \tag{87}$$

Next for the Jacobians between L3 and L2 from (4), we have the following interpolation relationship:

$$\begin{bmatrix}\boldsymbol{\theta}_t\\ \dot{\boldsymbol{\theta}}_t\\ \ddot{\boldsymbol{\theta}}_t\end{bmatrix} = \begin{bmatrix}\ddots & & \\ & \boldsymbol{\Lambda}_{ij} & \\ & & \ddots\end{bmatrix}\begin{bmatrix}\boldsymbol{\theta}_a\\ \dot{\boldsymbol{\theta}}_a\\ \ddot{\boldsymbol{\theta}}_a\end{bmatrix} + \begin{bmatrix}\ddots & & \\ & \boldsymbol{\Psi}_{ij} & \\ & & \ddots\end{bmatrix}\begin{bmatrix}\boldsymbol{\theta}_b\\ \dot{\boldsymbol{\theta}}_b\\ \ddot{\boldsymbol{\theta}}_b\end{bmatrix} \tag{88}$$

where $\boldsymbol{\Lambda}_{ij}, \boldsymbol{\Psi}_{ij} \in \mathbb{R}^{3\times 3}$ are some $3\times 3$ blocks of $\boldsymbol{\Lambda}(\tau)$ and $\boldsymbol{\Psi}(\tau)$. Thus, the Jacobians from L2 to L1 can be easily calculated as

$$\mathbf{J}_{\boldsymbol{\theta}_a^{(j)}}^{\boldsymbol{\theta}_t^{(i)}} = \boldsymbol{\Lambda}_{i,j},\quad \mathbf{J}_{\boldsymbol{\theta}_b^{(j)}}^{\boldsymbol{\theta}_t^{(i)}} = \boldsymbol{\Psi}_{i,j},\ i,j \in 0,1,2 \tag{89}$$

where $\boldsymbol{\theta}^{(0)} \triangleq \boldsymbol{\theta}$, $\boldsymbol{\theta}^{(1)} \triangleq \dot{\boldsymbol{\theta}}$, and $\boldsymbol{\theta}^{(2)} \triangleq \ddot{\boldsymbol{\theta}}$.

Finally for the Jacobian from L4 to L3, from the mappings (16), we have

$$\mathbf{J}_{\boldsymbol{\theta}_t}^{\mathbf{R}_t} = \frac{\partial \mathbf{R}_a \mathrm{Exp}(\boldsymbol{\theta}_t)}{\partial \boldsymbol{\theta}_t} = \boldsymbol{J}_r(\boldsymbol{\theta}_t) \tag{90}$$

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\theta}_t} = \frac{\partial \boldsymbol{J}_r(\boldsymbol{\theta}_t)\dot{\boldsymbol{\theta}}_t}{\partial \boldsymbol{\theta}_t} = \boldsymbol{H}_1(\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t) \tag{91}$$

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\dot{\boldsymbol{\theta}}_t} = \frac{\partial \boldsymbol{J}_r(\boldsymbol{\theta}_t)\dot{\boldsymbol{\theta}}_t}{\partial \dot{\boldsymbol{\theta}}_t} = \boldsymbol{J}_r(\boldsymbol{\theta}_t) \tag{92}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\theta}_t} = \boldsymbol{H}_1(\boldsymbol{\theta}_t, \ddot{\boldsymbol{\theta}}_t) + \boldsymbol{\mathcal{L}}_{11}(\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, \dot{\boldsymbol{\theta}}_t) \tag{93}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\dot{\boldsymbol{\theta}}_t} = \boldsymbol{H}_1(\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t) + \boldsymbol{\mathcal{L}}_{12}(\boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, \dot{\boldsymbol{\theta}}_t) \tag{94}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\ddot{\boldsymbol{\theta}}_t} = \boldsymbol{J}_r(\boldsymbol{\theta}_t). \tag{95}$$

Now, we can find the Jacobians from L3 to L1 by "merging" the L3-L2 and L2-L1 Jacobians as follows:

$$\mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\mathbf{R}_a} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\theta}_b}\mathbf{J}^{\boldsymbol{\theta}_b}_{\mathbf{R}_a} + \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\dot{\boldsymbol{\theta}}_b}\mathbf{J}^{\dot{\boldsymbol{\theta}}_b}_{\mathbf{R}_a} + \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\ddot{\boldsymbol{\theta}}_b}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_b}_{\mathbf{R}_a} \tag{96}$$

$$\mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\mathbf{R}_b} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\theta}_b}\mathbf{J}^{\boldsymbol{\theta}_b}_{\mathbf{R}_b} + \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\dot{\boldsymbol{\theta}}_b}\mathbf{J}^{\dot{\boldsymbol{\theta}}_b}_{\mathbf{R}_b} + \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\ddot{\boldsymbol{\theta}}_b}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_b}_{\mathbf{R}_b} \tag{97}$$

$$\mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\omega}_a} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\dot{\boldsymbol{\theta}}_a}\mathbf{J}^{\dot{\boldsymbol{\theta}}_a}_{\boldsymbol{\omega}_a},\ \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\omega}_b} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\dot{\boldsymbol{\theta}}_b}\mathbf{J}^{\dot{\boldsymbol{\theta}}_b}_{\boldsymbol{\omega}_b} + \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\ddot{\boldsymbol{\theta}}_b}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_b}_{\boldsymbol{\omega}_b} \tag{98}$$

$$\mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\alpha}_a} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\ddot{\boldsymbol{\theta}}_a}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_a}_{\boldsymbol{\alpha}_a},\ \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\boldsymbol{\alpha}_b} = \mathbf{J}^{\boldsymbol{\theta}_t^{(i)}}_{\ddot{\boldsymbol{\theta}}_b}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_b}_{\boldsymbol{\alpha}_b},\ i \in \{0,1,2\}. \tag{99}$$

Moving on to L4, we note that there is one direct path from L4 to L1, which corresponds to the Jacobian

$$\frac{\partial \mathbf{R}_a \mathrm{Exp}(\phi)}{\partial \mathbf{R}_a}|_{\phi=\boldsymbol{\theta}_t} = \mathbf{Ad}^{-1}_{\mathrm{Exp}(\boldsymbol{\theta}_t)} = \mathrm{Exp}(-\boldsymbol{\theta}_t). \tag{100}$$

Finally, we can assemble the L4-L1 Jacobians from (100) and the L4-L3 and L3-L1 Jacobians above. For $\mathbf{R}_t$, we have

$$\mathbf{J}^{\mathbf{R}_t}_{\mathbf{R}_a} = \mathrm{Exp}(-\boldsymbol{\theta}_t) + \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\mathbf{R}_a},\ \mathbf{J}^{\mathbf{R}_t}_{\mathbf{R}_b} = \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\mathbf{R}_b} \tag{101}$$

$$\mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\omega}_a} = \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\alpha}_a},\ \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\omega}_b} = \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\omega}_b} \tag{102}$$

$$\mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\alpha}_a} = \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\alpha}_a},\ \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\alpha}_b} = \mathbf{J}^{\mathbf{R}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\alpha}_b} \tag{103}$$

and similarly, for $\boldsymbol{\omega}_t$ and $\boldsymbol{\alpha}_t$

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\mathbf{R}_j} = \mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\mathbf{R}_j} + \mathbf{J}^{\boldsymbol{\omega}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\mathbf{R}_j} \tag{104}$$

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\omega}_j} = \mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\omega}_j} + \mathbf{J}^{\boldsymbol{\omega}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\boldsymbol{\omega}_j} \tag{105}$$

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\alpha}_j} = \mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\alpha}_j} + \mathbf{J}^{\boldsymbol{\omega}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\boldsymbol{\alpha}_j} \tag{106}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\mathbf{R}_i} = \mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\mathbf{R}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\mathbf{R}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\ddot{\boldsymbol{\theta}}_t}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_t}_{\mathbf{R}_i} \tag{107}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\omega}_i} = \mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\omega}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\boldsymbol{\omega}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\ddot{\boldsymbol{\theta}}_t}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_t}_{\boldsymbol{\omega}_i} \tag{108}$$

$$\mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\alpha}_i} = \mathbf{J}^{\boldsymbol{\alpha}_t}_{\boldsymbol{\theta}_t}\mathbf{J}^{\boldsymbol{\theta}_t}_{\boldsymbol{\alpha}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\dot{\boldsymbol{\theta}}_t}\mathbf{J}^{\dot{\boldsymbol{\theta}}_t}_{\boldsymbol{\alpha}_i} + \mathbf{J}^{\boldsymbol{\alpha}_t}_{\ddot{\boldsymbol{\theta}}_t}\mathbf{J}^{\ddot{\boldsymbol{\theta}}_t}_{\boldsymbol{\alpha}_i},\ i \in \{a,b\}. \tag{109}$$

To be completed, the Jacobian of the interpolated $\mathbb{R}^3$ states $\mathbf{p}_t$, $\mathbf{v}_t$, and $\mathbf{a}_t$ over $\mathbf{p}_a$, $\mathbf{v}_a$, and $\mathbf{a}_a$ and $\mathbf{p}_b$, $\mathbf{v}_b$, and $\mathbf{a}_b$ resembles (89)

$$\mathbf{J}^{\mathbf{p}_t^{(i)}}_{\mathbf{p}_a^{(j)}} = \boldsymbol{\Lambda}_{i,j},\quad \mathbf{J}^{\mathbf{p}_t^{(i)}}_{\mathbf{p}_b^{(j)}} = \boldsymbol{\Psi}_{i,j},\ i,j \in 0,1,2 \tag{110}$$

where $\mathbf{p}^{(0)} = \mathbf{p}$, $\mathbf{p}^{(1)} = \mathbf{v}$, and $\mathbf{p}^{(2)} = \mathbf{a}$.

### C2 SE(3) *Representation*

As can be seen in Fig. 2(b), the connections from layers L4 to L1 mirror the SO(3) case. Hence, the Jacobians from variables of L4 w.r.t. L1 can be directly adapted from those in Section IV-A. Specifically, to obtain these Jacobians, the readers can, respectively, replace the variables $\boldsymbol{\theta}$, $\dot{\boldsymbol{\theta}}$, $\ddot{\boldsymbol{\theta}}$, $\mathbf{R}$, $\boldsymbol{\omega}$, $\boldsymbol{\alpha}$, $\boldsymbol{J}_r$, $\boldsymbol{J}_r^{-1}$, $H$, $H'$, $L$, and $L'$ in the SO(3) case by $\boldsymbol{\xi}$, $\dot{\boldsymbol{\xi}}$, $\ddot{\boldsymbol{\xi}}$, $\mathbf{T}$, $\boldsymbol{\tau}$, $\dot{\boldsymbol{\tau}}$, $\boldsymbol{\mathcal{J}}_r$, $\boldsymbol{\mathcal{J}}_r^{-1}$, $\boldsymbol{\mathcal{H}}$, $\boldsymbol{\mathcal{H}}'$, $\boldsymbol{\mathcal{L}}$, and $\boldsymbol{\mathcal{L}}'$, whereas $\boldsymbol{\mathcal{L}}$ and $\boldsymbol{\mathcal{L}}'$ are defined as

$$\begin{aligned}\boldsymbol{\mathcal{L}}_{11}(\boldsymbol{\xi},\dot{\boldsymbol{\xi}},\dot{\boldsymbol{\xi}}) &= \frac{\partial}{\partial \boldsymbol{\xi}}\left[\boldsymbol{\mathcal{H}}_1(\boldsymbol{\xi},\dot{\boldsymbol{\xi}})\dot{\boldsymbol{\xi}}\right] \\ &= \begin{bmatrix} \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}}\right] & 0 \\ \frac{\partial}{\partial \boldsymbol{\theta}}\begin{bmatrix}\boldsymbol{S}_1(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\rho}})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{S}_2(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\rho}}\end{bmatrix} & \frac{\partial}{\partial \boldsymbol{\rho}}\begin{bmatrix}\boldsymbol{S}_1(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{S}_2(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\dot{\boldsymbol{\rho}}\end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} \boldsymbol{\mathcal{L}}_{11}(\boldsymbol{\theta},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\theta}}) & 0 \\ \begin{bmatrix}\boldsymbol{C}_{11}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\theta}}) \\ +\boldsymbol{\mathcal{L}}_{11}(\boldsymbol{\theta},\dot{\boldsymbol{\rho}},\dot{\boldsymbol{\theta}}) \\ \boldsymbol{C}_{21}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\rho}})\end{bmatrix} & \begin{bmatrix}\boldsymbol{C}_{12}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\theta}}) \\ +\boldsymbol{C}_{22}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\rho}})\end{bmatrix}, \end{bmatrix}\end{aligned} \tag{111}$$

$$\begin{aligned}\boldsymbol{\mathcal{L}}_{12}(\boldsymbol{\xi},\dot{\boldsymbol{\xi}},\dot{\boldsymbol{\xi}}) &= \frac{\partial}{\partial \dot{\boldsymbol{\xi}}}\left[\boldsymbol{\mathcal{H}}_1(\boldsymbol{\xi},\dot{\boldsymbol{\xi}})\boldsymbol{W}\right]\Big|_{\boldsymbol{W}=(\boldsymbol{W}_1,\boldsymbol{W}_2)=\dot{\boldsymbol{\xi}}} \\ &= \begin{bmatrix} \frac{\partial}{\partial \dot{\boldsymbol{\theta}}}\left[\boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\theta}})\boldsymbol{W}_1\right] & 0 \\ \frac{\partial}{\partial \dot{\boldsymbol{\theta}}}\begin{bmatrix}\boldsymbol{S}_1(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\boldsymbol{W}_1 \\ +\boldsymbol{S}_2(\boldsymbol{\xi},\dot{\boldsymbol{\theta}})\boldsymbol{W}_2\end{bmatrix} & \frac{\partial}{\partial \dot{\boldsymbol{\rho}}}\left[\boldsymbol{H}_1(\boldsymbol{\theta},\dot{\boldsymbol{\rho}})\dot{\boldsymbol{\theta}}\right] \end{bmatrix}\Big|_{\boldsymbol{W}=\dot{\boldsymbol{\xi}}} \\ &= \begin{bmatrix} L_{12}(\boldsymbol{\theta},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\theta}}) & 0 \\ \boldsymbol{C}_{13}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\theta}}) + \boldsymbol{C}_{23}(\boldsymbol{\xi},\dot{\boldsymbol{\theta}},\dot{\boldsymbol{\rho}}) & L_{12}(\boldsymbol{\theta},\dot{\boldsymbol{\rho}},\dot{\boldsymbol{\theta}}). \end{bmatrix}\end{aligned} \tag{112}$$

$$\begin{aligned}\boldsymbol{\mathcal{L}}'_{11}(\boldsymbol{\xi},\boldsymbol{\tau},\dot{\boldsymbol{\xi}}) &= \frac{\partial}{\partial \boldsymbol{\xi}}\left[\boldsymbol{\mathcal{H}}'_1(\boldsymbol{\xi},\boldsymbol{\tau})\dot{\boldsymbol{\xi}}\right] \\ &= \begin{bmatrix} \frac{\partial}{\partial \boldsymbol{\theta}}\left[\boldsymbol{H}'_1(\boldsymbol{\theta},\boldsymbol{\omega})\dot{\boldsymbol{\theta}}\right] & 0 \\ \frac{\partial}{\partial \boldsymbol{\theta}}\begin{bmatrix}\boldsymbol{S}'_1(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{H}'_1(\boldsymbol{\theta},\boldsymbol{\nu})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{S}'_2(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\rho}}\end{bmatrix} & \frac{\partial}{\partial \boldsymbol{\rho}}\begin{bmatrix}\boldsymbol{S}'_1(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{S}'_2(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\rho}}\end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} \boldsymbol{\mathcal{L}}'_{11}(\boldsymbol{\theta},\boldsymbol{\omega},\dot{\boldsymbol{\theta}}) & 0 \\ \begin{bmatrix}\boldsymbol{C}'_{11}(\boldsymbol{\xi},\boldsymbol{\omega},\dot{\boldsymbol{\theta}}) \\ +\boldsymbol{\mathcal{L}}'_{11}(\boldsymbol{\theta},\boldsymbol{\nu},\dot{\boldsymbol{\theta}}) \\ +\boldsymbol{C}'_{21}(\boldsymbol{\xi},\boldsymbol{\omega},\dot{\boldsymbol{\rho}})\end{bmatrix} & \begin{bmatrix}\boldsymbol{C}'_{12}(\boldsymbol{\xi},\boldsymbol{\omega},\dot{\boldsymbol{\theta}}) \\ +\boldsymbol{C}'_{22}(\boldsymbol{\xi},\boldsymbol{\omega},\dot{\boldsymbol{\rho}})\end{bmatrix} \end{bmatrix}\end{aligned} \tag{113}$$

$$\begin{aligned}\boldsymbol{\mathcal{L}}_{12}(\boldsymbol{\xi},\boldsymbol{\tau},\dot{\boldsymbol{\xi}}) &= \frac{\partial}{\partial \boldsymbol{\tau}}\left[\boldsymbol{\mathcal{H}}'_1(\boldsymbol{\xi},\boldsymbol{\tau})\dot{\boldsymbol{\xi}}\right] \\ &= \begin{bmatrix} \frac{\partial}{\partial \boldsymbol{\omega}}\left[\boldsymbol{H}'_1(\boldsymbol{\theta},\boldsymbol{\omega})\dot{\boldsymbol{\theta}}\right] & 0 \\ \frac{\partial}{\partial \boldsymbol{\omega}}\begin{bmatrix}\boldsymbol{S}'_1(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\theta}} \\ +\boldsymbol{S}'_2(\boldsymbol{\xi},\boldsymbol{\omega})\dot{\boldsymbol{\rho}}\end{bmatrix} & \frac{\partial}{\partial \boldsymbol{\nu}}\left[\boldsymbol{H}'_1(\boldsymbol{\theta},\boldsymbol{\nu})\dot{\boldsymbol{\theta}}\right] \end{bmatrix}\end{aligned}$$

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

$$= \begin{bmatrix} \boldsymbol{\mathcal{L}'}_{12}(\boldsymbol{\theta}, \boldsymbol{\omega}, \dot{\boldsymbol{\theta}}) & 0 \\ \boldsymbol{C'}_{13}(\boldsymbol{\xi}, \boldsymbol{\omega}, \dot{\boldsymbol{\theta}}) + \boldsymbol{C'}_{23}(\boldsymbol{\xi}, \boldsymbol{\omega}, \dot{\boldsymbol{\rho}}) & \boldsymbol{\mathcal{L}'}_{12}(\boldsymbol{\theta}, \boldsymbol{\nu}, \dot{\boldsymbol{\theta}}) \end{bmatrix}. \quad (114)$$

The matrices $\boldsymbol{S}, \boldsymbol{C}, \boldsymbol{S}'$, and $\boldsymbol{C}'$ are calculated using the procedure in Section IV-C.

Then, we find the Jacobians from layer L5 to L4 based on the following mappings:

$$\mathbf{R}_t = \mathbf{M}\mathbf{T}_t\mathbf{M}^\top,\ \mathbf{M} = [\mathbf{I}_{3\times3}\ \mathbf{0}_{3\times1}] \quad (115)$$

$$\mathbf{p}_t = \mathbf{M}\mathbf{T}_t\mathbf{N},\ \mathbf{N} = [\mathbf{0}_{3\times3},\ \mathbf{1}_{3\times1}]^\top$$

$$\boldsymbol{\omega}_t = \mathbf{U}\boldsymbol{\tau}_t,\ \mathbf{U} = [\mathbf{I}_{3\times3},\ \mathbf{0}_{3\times3}] \quad (116)$$

$$\mathbf{v}_t = \mathbf{R}_t\mathbf{D}\boldsymbol{\tau}_t = \mathbf{R}_t\boldsymbol{\nu}_t,\ \mathbf{D} = [\mathbf{0}_{3\times3},\ \mathbf{I}_{3\times3}] \quad (117)$$

$$\boldsymbol{\alpha}_t = \mathbf{U}\dot{\boldsymbol{\tau}}_t \quad (118)$$

$$\mathbf{a}_t = \mathbf{R}_t\mathbf{D}\dot{\boldsymbol{\tau}}_t + \mathbf{R}_t\boldsymbol{\omega}_t^\wedge\mathbf{D}\boldsymbol{\tau}_t = \mathbf{R}_t\boldsymbol{\beta}_t + \mathbf{R}_t\boldsymbol{\omega}_t^\wedge\boldsymbol{\nu}_t \quad (119)$$

which provides the following Jacobians between L5 and L4

$$\mathbf{J}^{\mathbf{R}_t}_{\mathbf{T}_t} = \mathbf{U},\ \mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\tau}_t} = \mathbf{U},\ \mathbf{J}^{\boldsymbol{\alpha}_t}_{\dot{\boldsymbol{\tau}}_t} = \mathbf{U}$$

$$\mathbf{J}^{\mathbf{p}_t}_{\mathbf{T}_t} = \mathbf{R}_t\mathbf{D},\ \mathbf{J}^{\mathbf{v}_t}_{\boldsymbol{\tau}_t} = \mathbf{R}_t\mathbf{D},\ \mathbf{J}^{\mathbf{a}_t}_{\dot{\boldsymbol{\tau}}_t} = \mathbf{R}_t\mathbf{D} \quad (120)$$

$$\mathbf{J}^{\mathbf{v}_t}_{\mathbf{R}_t} = -\mathbf{R}_t\boldsymbol{\nu}_t^\wedge \rightarrow \mathbf{J}^{\mathbf{v}_t}_{\mathbf{T}_t} = \mathbf{J}^{\mathbf{v}_t}_{\mathbf{R}_t}\mathbf{J}^{\mathbf{R}_t}_{\mathbf{T}_t}$$

$$\mathbf{J}^{\mathbf{a}_t}_{\mathbf{R}_t} = -\mathbf{R}_t\boldsymbol{\beta}_t^\wedge - \mathbf{R}_t(\boldsymbol{\omega}_t^\wedge\boldsymbol{\nu}_t)^\wedge \rightarrow \mathbf{J}^{\mathbf{a}_t}_{\mathbf{T}_t} = \mathbf{J}^{\mathbf{a}_t}_{\mathbf{R}_t}\mathbf{J}^{\mathbf{R}_t}_{\mathbf{T}_t}$$

$$\mathbf{J}^{\mathbf{a}_t}_{\boldsymbol{\omega}_t} = \mathbf{J}^{\mathbf{v}_t}_{\mathbf{R}_t}, \rightarrow \mathbf{J}^{\mathbf{a}_t}_{\boldsymbol{\tau}_t} = \mathbf{J}^{\mathbf{a}_t}_{\boldsymbol{\omega}_t}\mathbf{J}^{\boldsymbol{\omega}_t}_{\boldsymbol{\tau}_t} + \mathbf{R}_t\boldsymbol{\omega}_t^\wedge\mathbf{D}. \quad (121)$$

Similarly, between L1 and L0, we have the mappings

$$\mathbf{T}_i = \begin{bmatrix} \mathbf{R}_i & \mathbf{p}_i \\ \mathbf{0} & 1 \end{bmatrix},\ \boldsymbol{\tau}_i = \begin{bmatrix} \boldsymbol{\omega}_i \\ \mathbf{R}_i^\top\mathbf{v}_i \end{bmatrix}$$

$$\dot{\boldsymbol{\tau}}_i = \begin{bmatrix} \boldsymbol{\alpha}_i \\ \mathbf{R}_i^\top\mathbf{a}_i - \boldsymbol{\omega}_i^\wedge\mathbf{R}_i^\top\mathbf{v}_i \end{bmatrix} \quad (122)$$

which provide the following Jacobians:

$$\mathbf{J}^{\mathbf{T}_i}_{\mathbf{R}_i} = \mathbf{U}^\top,\ \mathbf{J}^{\mathbf{T}_i}_{\mathbf{p}_i} = \mathbf{D}^\top\mathbf{R}_i^\top \quad (123)$$

$$\mathbf{J}^{\boldsymbol{\tau}_i}_{\boldsymbol{\omega}_i} = \mathbf{U}^\top,\ \mathbf{J}^{\boldsymbol{\tau}_i}_{\mathbf{R}_i} = \mathbf{D}^\top(\mathbf{R}_i^\top\mathbf{v}_i)^\wedge,\ \mathbf{J}^{\boldsymbol{\tau}_i}_{\mathbf{v}_i} = \mathbf{D}^\top\mathbf{R}_i^\top \quad (124)$$

$$\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\boldsymbol{\alpha}_i} = \mathbf{U}^\top,\ \mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{R}_i} = \mathbf{D}^\top\left[(\mathbf{R}_i^\top\mathbf{a}_i)^\wedge - \boldsymbol{\omega}_i^\wedge(\mathbf{R}_i^\top\mathbf{v}_i)^\wedge\right] \quad (125)$$

$$\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{a}_i} = \mathbf{D}^\top\mathbf{R}_i^\top,\ \mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\boldsymbol{\omega}_i} = \mathbf{J}^{\boldsymbol{\tau}_i}_{\mathbf{R}_i},\ \mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{v}_i} = -\mathbf{D}^\top\boldsymbol{\omega}_i^\wedge\mathbf{R}_i^\top. \quad (126)$$

Hence, we can combine L4-L1 and L1-L0 Jacobians to obtain L4-L0 Jacobians. Specifically for $i \in \{a, b\}$, we have

$$\mathbf{J}^{\mathcal{X}_t}_{\mathbf{R}_i} = \mathbf{J}^{\mathcal{X}_t}_{\mathbf{T}_i}\mathbf{J}^{\mathbf{T}_i}_{\mathbf{R}_i} + \mathbf{J}^{\mathcal{X}_t}_{\boldsymbol{\tau}_i}\mathbf{J}^{\boldsymbol{\tau}_i}_{\mathbf{R}_i} + \mathbf{J}^{\mathcal{X}_t}_{\dot{\boldsymbol{\tau}}_i}\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{R}_i},\ \mathbf{J}^{\mathcal{X}_t}_{\mathbf{p}_i} = \mathbf{J}^{\mathcal{X}_t}_{\mathbf{T}_i}\mathbf{J}^{\mathbf{T}_i}_{\mathbf{p}_i} \quad (127)$$

$$\mathbf{J}^{\mathcal{X}_t}_{\boldsymbol{\omega}_i} = \mathbf{J}^{\mathcal{X}_t}_{\boldsymbol{\tau}_i}\mathbf{J}^{\boldsymbol{\tau}_i}_{\boldsymbol{\omega}_i} + \mathbf{J}^{\mathcal{X}_t}_{\dot{\boldsymbol{\tau}}_i}\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\boldsymbol{\omega}_i},\ \mathbf{J}^{\mathcal{X}_t}_{\mathbf{v}_i} = \mathbf{J}^{\mathcal{X}_t}_{\boldsymbol{\tau}_i}\mathbf{J}^{\boldsymbol{\tau}_i}_{\mathbf{v}_i} + \mathbf{J}^{\mathcal{X}_t}_{\dot{\boldsymbol{\tau}}_i}\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{v}_i} \quad (128)$$

$$\mathbf{J}^{\mathcal{X}_t}_{\boldsymbol{\alpha}_i} = \mathbf{J}^{\mathcal{X}_t}_{\dot{\boldsymbol{\tau}}_i}\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\boldsymbol{\alpha}_i},\ \mathbf{J}^{\mathcal{X}_t}_{\mathbf{a}_i} = \mathbf{J}^{\mathcal{X}_t}_{\dot{\boldsymbol{\tau}}_i}\mathbf{J}^{\dot{\boldsymbol{\tau}}_i}_{\mathbf{a}_i}, \mathcal{X}_t \in \{\mathbf{T}_t, \boldsymbol{\tau}_t, \dot{\boldsymbol{\tau}}_t\}. \quad (129)$$

Finally, we can prepend the L5-L4 Jacobians to the L4-L0 to obtain the L5-L0 Jacobians. For $\mathcal{Y}_i \in$ L0, we have

$$\mathbf{J}^{\mathcal{X}_t}_{\mathcal{Y}_i} = \mathbf{J}^{\mathcal{X}_t}_{\mathbf{T}_t}\mathbf{J}^{\mathbf{T}_t}_{\mathcal{Y}_i},\ \mathcal{X}_i \in \{\mathbf{R}_i, \mathbf{p}_i\} \quad (130)$$

$$\mathbf{J}^{\boldsymbol{\omega}_t}_{\mathcal{Y}_i} = \mathbf{J}^{\boldsymbol{\omega}_t}_{\mathbf{T}_t}\mathbf{J}^{\mathbf{T}_t}_{\mathcal{Y}_i},\ \mathbf{J}^{\boldsymbol{\alpha}_t}_{\mathcal{Y}_i} = \mathbf{J}^{\boldsymbol{\alpha}_t}_{\mathbf{T}_t}\mathbf{J}^{\mathbf{T}_t}_{\mathcal{Y}_i} \quad (131)$$

$$\mathbf{J}^{\mathbf{v}_t}_{\mathcal{Y}_i} = \mathbf{J}^{\mathbf{v}_t}_{\mathbf{T}_t}\mathbf{J}^{\mathbf{T}_t}_{\mathcal{Y}_i} + \mathbf{J}^{\mathbf{v}_t}_{\boldsymbol{\tau}_t}\mathbf{J}^{\boldsymbol{\tau}_t}_{\mathcal{Y}_i} \quad (132)$$

$$\mathbf{J}^{\mathbf{a}_t}_{\mathcal{Y}_i} = \mathbf{J}^{\mathbf{a}_t}_{\mathbf{T}_t}\mathbf{J}^{\mathbf{T}_t}_{\mathcal{Y}_i} + \mathbf{J}^{\mathbf{a}_t}_{\boldsymbol{\tau}_t}\mathbf{J}^{\boldsymbol{\tau}_t}_{\mathcal{Y}_i} + \mathbf{J}^{\mathbf{a}_t}_{\dot{\boldsymbol{\tau}}_t}\mathbf{J}^{\dot{\boldsymbol{\tau}}_t}_{\mathcal{Y}_i}. \quad (133)$$

## REFERENCES


[1] G. Cioffi, T. Cieslewski, and D. Scaramuzza, "Continuous-time vs. discrete-time vision-based SLAM: A comparative study," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2399–2406, Apr. 2022.

[2] W. Talbot et al., "Continuous-time state estimation methods in robotics: A survey," *IEEE Trans. Robot.*, vol. 41, pp. 4975–4999, 2025.

[3] M. Bosse and R. Zlot, "Continuous 3D scan-matching with a spinning 2D laser," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2009, pp. 4312–4319.

[4] M. Bosse, R. Zlot, and P. Flick, "Zebedee: Design of a spring-mounted 3-D range sensor with application to mobile mapping," *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1104–1119, Oct. 2012.

[5] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "CT-ICP: Real-time elastic LiDAR odometry with loop closure," in *Proc. Int. Conf. Robot. Automat. (ICRA)*, 2022, pp. 5580–5586.

[6] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, "NTU VIRAL: A visual-inertial-ranging-LiDAR dataset, from an aerial vehicle viewpoint," *Int. J. Robot. Res.*, vol. 41, no. 3, pp. 270–280, 2022.

[7] T.-M. Nguyen, D. Duberg, P. Jensfelt, S. Yuan, and L. Xie, "SLICT: Multi-input multi-scale surfel-based LiDAR-inertial continuous-time odometry and mapping," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 2102–2109, Apr. 2023.

[8] P. Furgale, T. D. Barfoot, and G. Sibley, "Continuous-time batch estimation using temporal basis functions," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2012, pp. 2088–2095.

[9] P. Furgale, J. Rehder, and R. Siegwart, "Unified temporal and spatial calibration for multi-sensor systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2013, pp. 1280–1286.

[10] J. Rehder, J. Nikolic, T. Schneider, T. Hinzmann, and R. Siegwart, "Extending kalibr: Calibrating the extrinsics of multiple IMUs and of individual axes," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2016, pp. 4304–4311.

[11] L. Oth, P. Furgale, L. Kneip, and R. Siegwart, "Rolling shutter camera calibration," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 1360–1367.

[12] A. Patron-Perez, S. Lovegrove, and G. Sibley, "A spline-based trajectory representation for sensor fusion and rolling shutter cameras," *Int. J. Comput. Vis.*, vol. 113, no. 3, pp. 208–219, 2015.

[13] E. Mueggler, G. Gallego, H. Rebecq, and D. Scaramuzza, "Continuous-time visual-inertial odometry for event cameras," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1425–1440, Dec. 2018.

[14] X. Lang, J. Lv, J. Huang, Y. Ma, Y. Liu, and X. Zuo, "Ctrl-VIO: Continuous-time visual-inertial odometry for rolling shutter cameras," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11537–11544, Oct. 2022.

[15] D. Hug and M. Chli, "HyperSLAM: A generic and modular approach to sensor fusion and simultaneous localization and mapping in continuous-time," in *Proc. Int. Conf. 3D Vis. (3DV)*, 2020, pp. 978–986.

[16] D. Hug, P. Bänninger, I. Alzugaray, and M. Chli, "Continuous-time stereo-inertial odometry," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6455–6462, Jul. 2022.

[17] D. Hug, I. Alzugaray, and M. Chli, "Hyperion–A fast, versatile symbolic Gaussian belief propagation framework for continuous-time SLAM," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 215–231.

[18] K. Li, Z. Cao, and U. D. Hanebeck, "Continuous-time ultra-wideband-inertial fusion," *IEEE Robot. Autom. Lett.*, vol. 8, no. 7, pp. 4338–4345, Jul. 2023.

[19] J. Quenzel and S. Behnke, "Real-time multi-adaptive-resolution-surfel 6D LiDAR odometry using continuous-time trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 5499–5506.

[20] J. Lv, K. Hu, J. Xu, Y. Liu, X. Ma, and X. Zuo, "CLINS: Continuous-time trajectory estimation for LiDAR-inertial system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 6657–6663.

[21] T.-M. Nguyen et al., "Eigen is all you need: Efficient LiDAR-inertial continuous-time odometry with internal association," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5330–5337, Jun. 2024.

[22] C. Park, P. Moghadam, J. L. Williams, S. Kim, S. Sridharan, and C. Fookes, "Elasticity meets continuous-time: Map-centric dense 3D LiDAR SLAM," *IEEE Trans. Robot.*, vol. 38, no. 2, pp. 978–997, Apr. 2022.


Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.

[23] C. H. Tong, P. Furgale, and T. D. Barfoot, "Gaussian process Gauss-Newton for non-parametric simultaneous localization and mapping," *Int. J. Robot. Res.*, vol. 32, no. 5, pp. 507–525, 2013.
[24] T. D. Barfoot, C. H. Tong, and S. Särkkä, "Batch continuous-time trajectory estimation as exactly sparse Gaussian process regression," in *Proc. Robot.: Sci. Syst.*, UC Berkeley, California, USA, 2014, vol. 10, pp. 1–10.
[25] S. Anderson, T. D. Barfoot, C. H. Tong, and S. Särkkä, "Batch nonlinear continuous-time trajectory estimation as exactly sparse Gaussian process regression," *Auton. Robots*, vol. 39, pp. 221–238, 2015.
[26] S. Anderson and T. D. Barfoot, "Full steam ahead: Exactly sparse Gaussian process regression for batch continuous-time trajectory estimation on SE(3)," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2015, pp. 157–164.
[27] T. D. Barfoot, *State Estimation for Robot.* Cambridge, U.K.: Cambridge Unive. Press, 2024.
[28] M. Seeger, "Gaussian processes for machine learning," *Int. J. Neural Syst.*, vol. 14, no. 2, pp. 69–106, 2004.
[29] J. Johnson, J. Mangelson, T. Barfoot, and R. Beard, "Continuous-time trajectory estimation: A comparative study between Gaussian process and spline-based approaches," 2024, *arXiv:2402.00399*.
[30] T. D. Barfoot and P. T. Furgale, "Associating uncertainty with three-dimensional poses for use in estimation problems," *IEEE Trans. Robot.*, vol. 30, no. 3, pp. 679–693, Jun. 2014.
[31] Y. Wu et al., "Picking up speed: Continuous-time LiDAR-only odometry using Doppler velocity measurements," *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 264–271, Jan. 2023.
[32] K. Burnett, A. P. Schoellig, and T. D. Barfoot, "Continuous-time radar-inertial and LiDAR-inertial odometry using a Gaussian process motion prior," *IEEE Trans. Robot.*, vol. 41, pp. 1059–1076, 2025.
[33] K. Burnett, A. P. Schoellig, and T. D. Barfoot, "IMU as an input vs. a measurement of the state in inertial-aided state estimation," 2024, *arXiv:2403.05968*.
[34] J. Peršić, L. Petrović, I. Marković, and I. Petrović, "Spatiotemporal multisensor calibration via Gaussian processes moving target tracking," *IEEE Trans. Robot.*, vol. 37, no. 5, pp. 1401–1415, Oct. 2021.
[35] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time Gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.
[36] X. Yan, V. Indelman, and B. Boots, "Incremental sparse GP regression for continuous-time trajectory estimation and mapping," *Robot. Auton. Syst.*, vol. 87, pp. 120–132, 2017.
[37] M. Mukadam, J. Dong, F. Dellaert, and B. Boots, "Simultaneous trajectory estimation and planning via probabilistic inference," in *Proc. Robot.: Sci. Syst.*, 2017, pp. 1–9.
[38] M. Warren, M. Paton, K. MacTavish, A. P. Schoellig, and T. D. Barfoot, "Towards visual teach and repeat for GPS-denied flight of a fixed-wing UAV," in *Proc. Field Serv. Robot.: Results 11th Int. Conf.*, 2018, pp. 481–498.
[39] H. Ovrén and P.-E. Forssén, "Trajectory representation and landmark projection for continuous-time structure from motion," *Int. J. Robot. Res.*, vol. 38, no. 6, pp. 686–701, 2019.
[40] A. Haarbach, T. Birdal, and S. Ilic, "Survey of higher order rigid body motion interpolation methods for keyframe animation and continuous-time trajectory estimation," in *Proc. 2018 Int. Conf. 3D Vis. (3DV)*, 2018, pp. 381–389.
[41] X. Zheng and J. Zhu, "Traj-LIO: A resilient multi-LiDAR multi-IMU state estimator through sparse Gaussian process," 2024, *arXiv:2402.09189*.
[42] T. Y. Tang, D. J. Yoon, and T. D. Barfoot, "A white-noise-on-jerk motion prior for continuous-time trajectory estimation on SE (3)," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 594–601, Apr. 2019.
[43] S. Agarwal and K. Mierle, "Ceres solver: Tutorial & reference," (n.d.). [Online]. Available: http://ceres-solver.org/
[44] J. Lv, X. Zuo, K. Hu, J. Xu, G. Huang, and Y. Liu, "Observability-aware intrinsic and extrinsic calibration of LiDAR-IMU systems," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3734–3753, Dec. 2022.
[45] J. Lv, X. Lang, J. Xu, M. Wang, Y. Liu, and X. Zuo, "Continuous-time fixed-lag smoothing for LiDAR-inertial-camera SLAM," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 4, pp. 2259–2270, Aug. 2023.
[46] X. Lang et al., "Coco-LIC: Continuous-time tightly-coupled LiDAR-inertial-camera odometry using non-uniform B-spline," *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7074–7081, Nov. 2023.
[47] X. Zheng and J. Zhu, "Traj-LO: In defense of LiDAR-only odometry using an effective continuous-time trajectory," *IEEE Robot. Autom. Lett.*, vol. 9, no. 2, pp. 1961–1968, Feb. 2024.
[48] J. Sola, J. Deray, and D. Atchuthan, "A micro Lie theory for state estimation in robotics," 2018, *arXiv:1812.01537*.
[49] T. D. Barfoot, "Integral forms in matrix lie groups," 2025, *arXiv:2503.02820*.
[50] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stueckler, and D. Cremers, "The TUM VI benchmark for evaluating visual-inertial odometry," in *Proc. Int. Conf. Intell. Robots Syst. (IROS)*, 2018, pp. 1680–1687.
[51] C. Sommer, V. Usenko, D. Schubert, N. Demmel, and D. Cremers, "Efficient derivative computation for cumulative B-splines on Lie groups," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11148–11156.
[52] V. Usenko, N. Demmel, and D. Cremers, "The double sphere camera model," in *Proc. Int. Conf. 3D Vis. (3DV)*, 2018, pp. 552–560.
[53] Z. Liu and F. Zhang, "BALM: Bundle adjustment for LiDAR mapping," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3184–3191, Apr. 2021.
[54] J. Li, T.-M. Nguyen, S. Yuan, and L. Xie, "PSS-BA: LiDAR bundle adjustment with progressive spatial smoothing," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024, pp. 1124–1129.
[55] W. Yu et al., "I2EKF-LO: A dual-iteration extended Kalman filter based LiDAR odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024, pp. 10453–10460.
[56] H. Shen et al., "CTE-MLO: Continuous-time and efficient multi-LiDAR odometry with localizability-aware point cloud sampling," *IEEE Trans. Field Robot.*, vol. 2, pp. 165–187, 2025.
[57] X. Liu, C. Yuan, and F. Zhang, "Targetless extrinsic calibration of multiple small FOV LiDARs and cameras using adaptive voxelization," *IEEE Trans. Instrum. Meas.*, vol. 71, 2022, Art. no. 8502612.
[58] K. Li, M. Li, and U. D. Hanebeck, "Towards high-performance solid-state-LiDAR-inertial odometry and mapping," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5167–5174, Jul. 2021.
[59] Y. Cai, W. Xu, and F. Zhang, "ikd-Tree: An incremental K-D tree for robotic applications," 2021, *arXiv:2102.10808*.
[60] C. Yuan, W. Xu, X. Liu, X. Hong, and F. Zhang, "Efficient and probabilistic adaptive voxel mapping for accurate online LiDAR odometry," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 8518–8525, Jul. 2022.
[61] D. Duberg and P. Jensfelt, "UFOMap: An efficient probabilistic 3D mapping framework that embraces the unknown," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6411–6418, Oct. 2020.
[62] J. Jiao, H. Ye, Y. Zhu, and M. Liu, "Robust odometry and mapping for multi-LiDAR systems with online extrinsic calibration," *IEEE Trans. Robot.*, vol. 38, no. 1, pp. 351–371, Feb. 2022.



**Thien-Minh Nguyen** (Member, IEEE) received the bachelor of engineering (honours) degree in automation from Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam, in 2014, and the Ph.D. degree in robotics from Nanyang Technological University (NTU), Singapore, in 2020.

He is currently a Lecturer (Assistant Professor) in robotics with the School of Mechanical and Mining Engineering, The University of Queensland, St Lucia, QLD, Australia. Prior to this, he was a Research Assistant Professor with NTU, and a Wallenberg-NTU Presidential Postdoctoral Fellow jointly hosted by NTU and KTH Royal Institute of Technology, Stockholm, Sweden. His research interests include multirobot systems, robot perception, navigation, and robot learning.

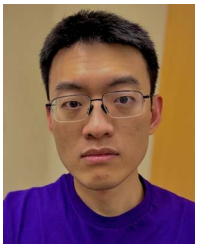
Dr. Nguyen was the recipient of the Class of 2020 Doctorate Innovation Award (Best Thesis Award) from the NTU School of EEE, the 2025 Science and Technology Golden Globe Award, the 2025 Outstanding Young Vietnamese Award, the Second Prize at the 2021 Hilti SLAM Challenge, and the Intel Vietnam Engineering Scholarship in 2012. He is an Associate Editor for IEEE ROBOTICS AND AUTOMATION LETTERS in the area of localization and mapping.



**Ziyu Cao** (Student Member, IEEE) received the B.Sc. degree in vehicle engineering from the Beijing Institute of Technology, Beijing, China, in 2019, and the M.Sc. degree in mechatronics and information technology from the Karlsruhe Institute of Technology, Karlsruhe, Germany, in 2022. She is currently working toward the Ph.D. degree in electrical engineering with focus on mobile robotics with Linköping University, Linköping, Sweden.

She conducted bachelor's thesis with Karlsruhe Institute of Technology. Her research interests include state estimation and multisensor fusion in mobile robotics.

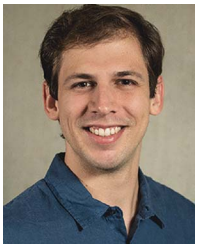
Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.



**Kailai Li** (Member, IEEE) received the B.E. degree in mechanical engineering and automation from Tsinghua University, Beijing, China, in 2014, the M.Sc. degree in automation engineering from RWTH Aachen University, Aachen, Germany, in 2017, and the Ph.D. degree in computer science from the Institute for Anthropomatics and Robotics, Karlsruhe Institute of Technology, Karlsruhe, Germany, in 2021.

Since 2024, he has been an Assistant Professor with the Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, Groningen, The Netherlands. From 2022 to 2024, he was a Postdoctoral Researcher with the Department of Electrical Engineering, Linköping University, Linköping, Sweden. He conducted the master's thesis with ETH Zürich, Zürich, Switzerland. His research interests include sensing, learning, and control for mobile robotics.

Dr. Li was the recipient of the Jean-Pierre Le Cadre Best Paper Award (first runner-up) by the International Society for Information Fusion in 2022. He is currently an Associate Editor for IEEE ROBOTICS AND AUTOMATION LETTERS in the area of visual perception and learning.



**William Talbot** (Student Member, IEEE) received the bachelor of engineering degree in mechatronic engineering with a major in space engineering and the bachelor of science degree in computer science with a minor in physics from the University of Sydney, Sydney, NSW, Australia, in 2023, and the master of science degree in robotics, systems, and control from ETH Zürich, Zürich, Switzerland, in 2024, where he is currently working toward the Ph.D. degree in robotics with the Robotic Systems Laboratory.

He conducted the undergraduate thesis with the NASA Jet Propulsion Laboratory. His research interests include state estimation, mapping, and physical modeling.



**Tongxing Jin** received the bachelor's degree in detection, guidance and control technology from Harbin Engineering University, Harbin, China, in 2016, and the master's degree in control engineering from the Harbin Institute of Technology, Harbin, in 2019. He is currently working toward the Ph.D. degree with Nanyang Technological University, Singapore.

His research interests include embodied perception and simultaneous localization and mapping (SLAM).



**Shenghai Yuan** (Member, IEEE) received the B.S. and Ph.D. degrees in electrical and electronic engineering from Nanyang Technological University, Singapore, in 2013 and 2019, respectively.

He is currently a Senior Research Fellow with the Centre for Advanced Robotics Technology Innovation, Nanyang Technological University, Singapore. His research interests include robot perception and navigation.

Dr. Yuan was the recipient of a number of awards, notably at the 2021 Hilti SLAM Challenge, the 2023 ICCV SLAM Challenge, the IROS 2023 Best Entertainment and Amusement Paper Award, and the Outstanding Reviewer Award at ICRA 2024. He is currently an Associate Editor for *Unmanned Systems* and IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING.



**Timothy D. Barfoot** (Fellow, IEEE) received the B.A.Sc. degree in engineering science and the Ph.D. degree in aerospace science and engineering from the University of Toronto, Toronto, ON, Canada, in 1997 and 2002, respectively.

He is currently a Professor with the University of Toronto Robotics Institute, Toronto. His research focuses on enabling robotic systems to operate robustly over long durations in large-scale, unstructured, 3-D environments using rich onboard sensing and computation. He has authored *State Estimation for Robotics* (Cambridge, 2017, 2024) and coauthored the SLAM Handbook (Cambridge, 2026). His research interests include guidance, navigation, and control of autonomous systems for a wide range of applications.

Dr. Barfoot is a Fellow of the Canadian Academy of Engineering. He is currently the Editor-in-Chief of IEEE TRANSACTIONS ON FIELD ROBOTICS.



**Lihua Xie** (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Newcastle, Callaghan, NSW, Australia, in 1992.

Since 1992, he has been with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, where he is currently the NTU President's Chair in control engineering. He was the Director of the Centre for Advanced Robotics Technology Innovation and Co-Director of the Delta-NTU Corporate Laboratory for Cyber-Physical Systems. His research interests include robust control and estimation, networked control systems, multiagent systems, and uncrewed systems.

Dr. Xie was an IEEE Distinguished Lecturer from 2012 to 2014. He is a Fellow of the Academy of Engineering Singapore, IFAC, and CAA. He is the Editor-in-Chief of *Unmanned Systems*. He was an Editor of the IET Book Series in Control and an Associate Editor for several journals, including IEEE TRANSACTIONS ON AUTOMATIC CONTROL, *Automatica*, and IEEE TRANSACTIONS ON CONTROL SYSTEMS TECHNOLOGY.

Authorized licensed use limited to: INSTITUTE OF AUTOMATION CAS. Downloaded on October 17,2026 at 03:07:28 UTC from IEEE Xplore. Restrictions apply.